PCCST 501 – COMPUTER NETWORKS

Ms. ANJU MARKOSE,
Asst. Professor & Dept. Secretary,
Dept. of Computer Science and Design,
VJCET, Vazhakulam.
Ph No: 9946004657.

MODULE I

INTRODUCTION TO COMPUTER NETWORKS, TCP/IP PROTOCOL SUITE AND APPLICATION LAYER

1.1 Introduction to Computer Networks

A computer network is a collection of interconnected devices — such as computers, servers, and mobile devices — that are linked together so that they can communicate with each other and share data, hardware, and software resources. The idea of "interconnection" is central to this definition: two or more devices must be physically or wirelessly connected through a communication medium before any exchange of information can take place. Without such a connection, each device would function as an isolated unit, unable to benefit from the resources or services available on other machines.

In simple terms, a computer network can also be defined as the interconnection of devices that are capable of exchanging information. This exchange may take the form of a text message between two people, a file transferred from one server to another, or a video stream delivered from a content provider to a user's smartphone. Regardless of the type of information exchanged, the underlying requirement remains the same: a reliable communication path between the sending and receiving devices.

Additional points to note: (a) A network can be as small as two computers connected by a single cable, or as large as the Internet, which links billions of devices worldwide. (b) Networks are built using a combination of hardware components (cables, routers, switches, network interface cards) and software components (protocols, operating system network services, and applications). (c) The performance of a network is generally judged using criteria such as performance (delay and throughput), reliability (failure rate and recovery time), and security (protection from unauthorized access).

Devices That Form a Network

A wide variety of devices can participate in a computer network. Common end-user devices include desktop computers, laptops, and smartphones, while servers provide centralized services such as file storage, e-mail, and web hosting to other devices on the network. In addition to end-user devices,

networks rely on intermediary devices such as printers (which are shared network resources), routers (which connect different networks together), and switches (which connect multiple devices within the same local network).

The connections between these devices can be established using two broad categories of transmission media. Wired media use physical cables, such as Ethernet cables (twisted-pair) or fiber-optic cables, to carry signals from one device to another; these media generally provide high speed and are less susceptible to interference. Wireless media, on the other hand, use radio waves or infrared signals — technologies such as Wi-Fi, Bluetooth, and cellular communication — to connect devices without a physical cable, offering mobility and ease of installation at the cost of being more vulnerable to interference and eavesdropping.

- Extra point: Hybrid networks are common in practice — a home network, for example, typically uses wired Ethernet for a desktop PC and a smart TV, while laptops and phones connect wirelessly to the same router.
- Extra point: The choice of medium is influenced by factors such as cost, distance, required speed, and the physical environment (e.g., fiber is preferred for long-distance backbone links, while Wi-Fi is preferred for short-range, mobile connectivity).

1.2 Why Do We Need Computer Networks?

Computer networks are not built merely for the sake of connecting machines together; they exist because interconnected systems make it possible to accomplish tasks that would be difficult, slow, or expensive to perform using isolated, stand-alone computers. The key motivations for building computer networks are resource sharing, communication, data sharing, and improved reliability and efficiency.

1. Resource Sharing

One of the most fundamental reasons for building a network is to allow multiple users to share hardware and software resources efficiently, instead of each user requiring a dedicated copy of every resource. For example, a single expensive printer can be shared by an entire office instead of every desk needing its own printer; files and folders stored on one machine can be accessed by authorized users elsewhere on the network; a single high-speed Internet connection can be shared among many devices in a home or office; and licensed software applications installed on a server can be accessed by many client machines simultaneously. This sharing reduces overall cost and avoids duplication of resources.

2. Communication

Networks enable fast and effective communication among users located in different places, ranging

include electronic mail (e-mail) for asynchronous messaging, video conferencing for real-time face-to-face meetings over long distances, instant messaging for quick text-based conversations, and voice and video calls that replace or supplement traditional telephone conversations. This communication capability has transformed the way businesses, educational institutions, and individuals interact.

3. Data Sharing

Beyond simple communication, networks facilitate the easy access, storage, and exchange of large volumes of information. Centralized databases allow multiple applications and users to query and update consistent data; cloud storage services let users store files remotely and retrieve them from any connected device; multimedia content such as images, audio, and video can be streamed or downloaded across the network; and shared documents allow multiple people to collaborate on the same file, often simultaneously.

4. Improved Reliability and Efficiency

Networked systems can enhance the overall performance, availability, and productivity of an organization's computing infrastructure. Backup and recovery systems can replicate critical data across multiple networked machines or sites so that a single point of failure does not result in permanent data loss. Distributed processing allows a large computational task to be divided among several networked computers, reducing the time required to complete it. Load sharing among devices prevents any single machine from becoming overloaded, and centralized management and maintenance allow administrators to monitor, update, and secure many machines from a single location, which is far more efficient than managing each computer individually.

- Extra point: Networks also support scalability — an organization can add new users, devices, or services without redesigning the entire computing infrastructure from scratch.
- Extra point: Networking underpins modern paradigms such as cloud computing, the Internet of Things (IoT), and remote work, all of which depend on always-on connectivity between distributed devices.

Example of a Small Network

A simple, everyday illustration of a computer network is a home Wi-Fi setup. When two or more devices at home are connected through a Wi-Fi router, they automatically form a small computer network. The Wi-Fi router acts as the central device that connects all the devices in the network; the laptop and phone can communicate with each other directly through the router; the router also provides Internet access to every connected device; and resources such as files, printers, and streaming media can be shared among all the devices on this small network.

Real-life example: A family uses a Wi-Fi router at home to connect a laptop, a smartphone, a smart t simultaneously and can also share

resources — for instance, streaming a movie from the laptop to the smart TV, or printing a document from the smartphone — through the same home network.

1.3 Components of a Computer Network

A computer network consists of three main categories of components: End Devices, Connecting Devices, and Transmission Media. Each of these categories plays a distinct role, and a functioning network requires all three to be present and correctly configured.

1. End Devices (Hosts)

End devices, also called hosts, are the devices that actually send, receive, or process data within a network. They represent the source and destination of most network traffic. Common examples include computers, laptops, smartphones, tablets, servers, and printers. The main functions of end devices are to generate and receive data, communicate with other devices on the network, and access network resources and services such as file shares, printers, or Internet-based applications.

2. Connecting Devices

Connecting devices are responsible for linking end devices and networks together and for directing data toward its correct destination. A router connects different networks to one another and provides access to external networks such as the Internet. A switch connects multiple devices within a single Local Area Network (LAN) and transfers data efficiently between them by forwarding frames only to the intended recipient port. A modem connects a network to an Internet Service Provider (ISP) and converts digital signals generated by computers into analog signals suitable for transmission over telephone or cable lines, and vice versa. An access point (AP) provides wireless connectivity to devices and extends the coverage area of a Wi-Fi network.

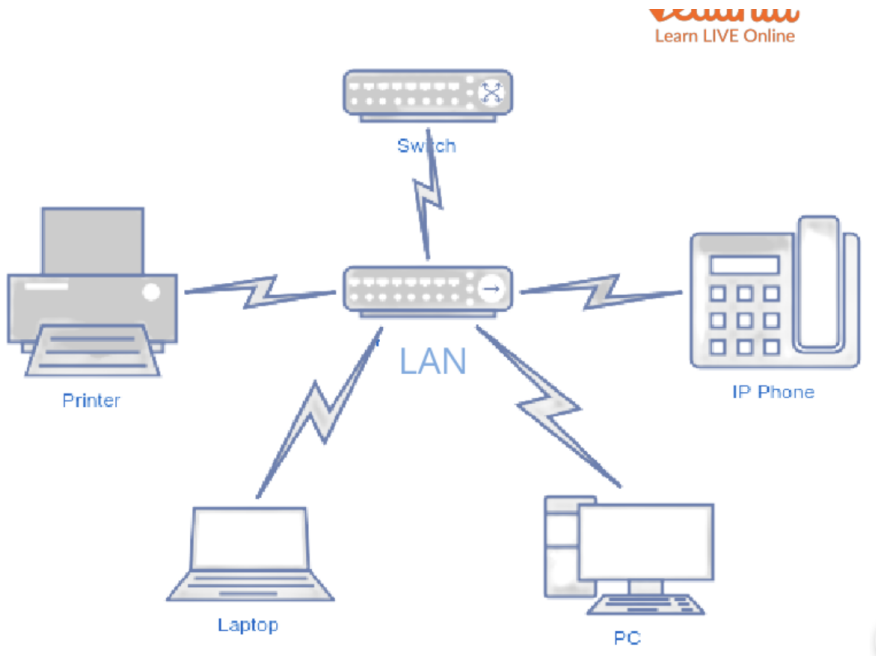


Figure: Examples of networking hardware — devices such as routers and switches that interconnect end devices and networks.

3. Transmission Media

Transmission media refer to the communication channel through which data physically travels from one device to another. Transmission media are broadly classified into wired and wireless categories.

A. Wired Media

Twisted pair cable is the most commonly used network cable and is widely used in Ethernet networks because it is inexpensive and easy to install. Coaxial cable is used in cable television networks and for broadband Internet access, offering better shielding against interference than twisted pair. Fiber optic cable uses light signals for transmission and offers very high speed and long-distance communication capability, making it the medium of choice for backbone and long-haul links.

B. Wireless Media

Radio waves are used for wireless communication over varying distances and form the basis for many wireless technologies. Wi-Fi enables wireless networking in homes, offices, and public places by using radio-frequency signals within a limited range. Satellite communication is used for long-distance communication and for providing connectivity to remote areas that lack terrestrial infrastructure.

- Extra point: The theoretical maximum data rate of a transmission medium is influenced by its bandwidth and the signal-to-noise ratio, as described by the Shannon capacity formula.
- Extra point: Attenuation (signal weakening over distance) and electromagnetic interference (EMI) affect wired media, while multipath fading and interference from other wireless devices affect wireless media.

1.4 Types of Computer Networks

Computer networks are classified according to their geographical coverage, ranging from a network spanning just a few meters to one that spans the entire globe. Understanding this classification helps in selecting the appropriate networking technology for a given scale of deployment.

TYPES OF NETWORK

Network Type	Full Form	Area of Coverage	Example
PAN	Personal Area Network	A few meters (around 1–10 m)	Bluetooth connection between a phone and earbuds
LAN	Local Area Network	A room, building, or campus (up to a few km)	Network within a school, office, or home
CAN	Campus Area Network	Multiple buildings within a campus (1–10 km)	University campus network
MAN	Metropolitan Area Network	A city or metropolitan area (10–100 km)	City-wide cable TV or internet network
WAN	Wide Area Network	Countries or continents (100 km and above)	Bank networks connecting branches worldwide
GAN	Global Area Network	Worldwide coverage	The Internet

A Personal Area Network (PAN) is the smallest type of network, typically connecting personal devices within a range of a few meters, such as a smartphone paired with wireless earbuds via Bluetooth. A Local Area Network (LAN) covers a room, a building, or a campus, and is the type of network most commonly encountered in homes, schools, and offices. A Campus Area Network (CAN) extends this concept to multiple buildings within the same campus, such as different departments of a university connected together. A Metropolitan Area Network (MAN) spans a city or metropolitan area, often used for city-wide cable television or Internet distribution. A Wide Area Network (WAN) covers countries or continents and is typically used by organizations such as banks to connect branches located far apart. Finally, a Global Area Network (GAN) provides worldwide coverage, with the Internet itself being the most prominent example of a GAN.

- Extra point: As the geographical coverage of a network increases, the technologies used also tend to change — LANs commonly use Ethernet or Wi-Fi, whereas WANs rely on leased lines, MPLS, or satellite links.
- Extra point: The boundary between these categories is not always rigid in practice; for example, a large enterprise LAN can sometimes be as large as a small MAN.

Intranet

An intranet is a private network that can only be accessed by authorized users. The prefix "intra" means "internal," and therefore an intranet is, by definition, designed for internal communications within an organization. An intranet is used within an organization to securely share information, resources, and communication among its employees. It uses the same underlying technologies as the public Internet (such as HTTP and TCP/IP) but is accessible only to authorized users within the organization, typically protected by firewalls and authentication systems.

Common examples of intranets include an employee portal used for HR-related tasks, an internal company website containing policies and announcements, and a school management system used by staff and students to manage academic records.

1.5 Overview of the Internet

What Is an Internetwork?

An internetwork (or "internet" with a lowercase "i") is formed when two or more networks are connected together so that devices on one network can communicate with devices on another network. These constituent networks may be LANs, WANs, or a combination of both. In essence, an internetwork is a collection of interconnected networks that behave, from a communication standpoint, as if they were a single unified system.

Internetworking is needed because a real-world organization rarely operates a single, isolated network; instead, it typically has multiple offices, departments, and locations, each with its own local network, that must be connected together to enable seamless communication and resource sharing across the entire organization. The Internet itself is not a single network but the largest and most well-known example of an internetwork — a network of networks — formed by interconnecting millions of private, public, academic, business, and government networks across the world.

Components of an Internetwork

An internetwork is typically composed of LANs, which provide communication within a local area; WANs, which connect distant networks across large geographical areas; routers, which connect different networks together and route packets between them based on logical (IP) addresses; and switches, which forward packets within a single LAN based on physical (MAC) addresses.

Role of Routers and Switches

A switch works inside a single LAN. It sends packets (technically, frames) to the correct destination host within that LAN and does not forward local traffic outside the LAN boundary. A router, in contrast, connects multiple networks together and routes packets from one network to another, making decisions based on the destination network address.

When a computer sends data to another computer on the same local network, the switch delivers the data directly to the destination device, and the router is not involved at all in this local exchange. However, when a computer sends data to a device on a different network — for example, a different office connected via a WAN — the data must travel through the local switch, then the local router, across the WAN, to a router on the destination network, and finally through a switch on that remote LAN before reaching the destination device. This complete path can be summarized as: Host → Switch → Router → WAN → Router → Switch → Host.

- Extra point: Routers operate at the Network layer (Layer 3) of the OSI model, while switches typically operate at the Data Link layer (Layer 2).
- Extra point: Modern "Layer 3 switches" blur this distinction by performing basic routing functions in hardware at very high speed.

Point-to-Point WAN and Switched WAN

A Point-to-Point WAN directly connects two devices or networks using a dedicated communication link, such as a leased cable or a wireless connection. A typical example is a dedicated connection between two branch offices of the same company, where the link is reserved exclusively for traffic between those two sites.

A Switched WAN, by contrast, connects multiple devices or networks through intermediate switches, allowing communication to take place over a much larger area involving many endpoints rather than just two. The Internet itself, along with large telecommunications networks, is a prime example of a switched WAN.

Figure 1.2 *A point-to-point WAN*

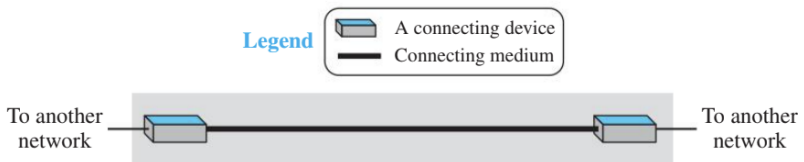


Figure: A Point-to-Point WAN directly linking two networks/devices through a dedicated line.

Figure 1.3 *A switched WAN*

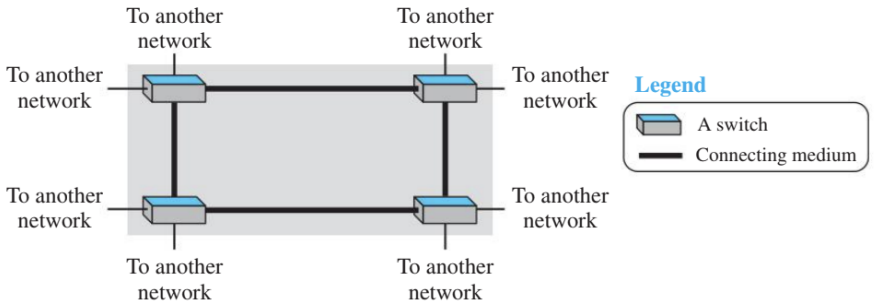


Figure: A Switched WAN connecting multiple networks/devices through intermediate switches.

Private Internet vs Internet

The term "private internet" (with a small "i") refers to a network created by connecting a company's or organization's internal networks together; it remains private and is accessible only to authorized users within that organization. The term "Internet" (with a capital "I") refers to the single, global public network that connects millions of networks and devices worldwide and is accessible to anyone with an Internet connection.

For example, a company's internal network connecting all of its offices constitutes a private internet, whereas the worldwide network used for websites, e-mail, and other online services is simply "the Internet."

1.6 Circuit Switching and Packet Switching

Circuit Switching

A circuit-switched network creates a dedicated communication path, called a circuit, between two devices before communication actually begins. Once this circuit is established, the entire path remains reserved exclusively for those two users until the communication session ends, regardless of whether data is actively being sent at every instant.

Consider a typical example: four telephones on each side of a network are connected through switches, and the two switches are connected by a single high-capacity line. This high-capacity line can support four simultaneous voice calls, and the switches simply forward the voice signals without storing any data.

In the case of full utilization, all four telephones on one side are connected to four telephones on the other side, so the high-capacity line carries four voice calls simultaneously, meaning the entire capacity of the line is used — this represents the most efficient situation for a circuit-switched network. In the

a given time, meaning only one out of

the four available channels is actually carrying traffic while the remaining three channels stay unused even though they remain reserved and available; in this scenario, only twenty-five percent of the line's capacity is being utilized. This example illustrates why circuit switching can become inefficient during periods of low traffic, since reserved capacity that is not being used cannot be given to other users.

Types of Switched Networks

There are two main types of switched networks:

1. **Circuit-Switched Network**
2. **Packet-Switched Network**

1. Circuit-Switched Network

Introduction

In a **circuit-switched network**, a dedicated communication path called a **circuit** is established between two end systems before communication begins.

The path remains reserved during the entire communication session.

Working Principle

- A fixed path is created between sender and receiver
- Resources are reserved for the whole communication period
- The connection can either be:
  - Active
  - Inactive

The switch only activates or deactivates the circuit.

Figure: Circuit-switched network — telephones connected through switches over a dedicated high-capacity line.

Example

Traditional telephone networks are classic examples of circuit switching.

```
Telephone ---- Switch ===== Switch ---- Telephone
```

Once a call is established:

- The communication path remains dedicated
- Other users cannot use that reserved capacity

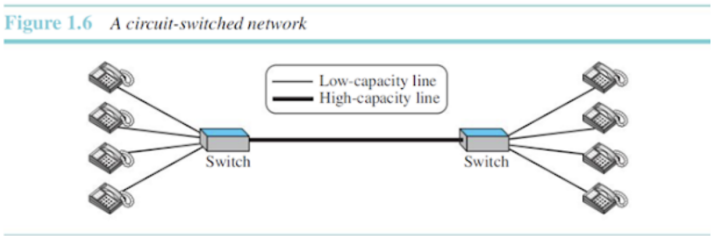


Figure: Circuit switching — illustration of full utilization versus partial utilization of the reserved line capacity.

Packet-Switched Network

In a packet-switched network, data is not sent as one continuous, uninterrupted stream. Instead, the data is divided into small units called packets. Each packet contains a portion of the overall data, travels independently through the network (potentially via different routes), and can be stored temporarily and forwarded later if the network is congested. In this architecture, computers act as end devices, while routers act as the switching elements; routers have memory in the form of queues that allow them to store packets temporarily when necessary.

In a low-traffic scenario, when a single computer at Site A communicates with a single computer at Site B, the communication line has more than enough capacity, so packets are forwarded immediately with no waiting or delay. In a heavy-traffic scenario, when many computers send packets at the same time, the communication line may reach its maximum capacity; new packets arriving at a router cannot be sent immediately, so they are placed in a queue (buffer) and forwarded one by one in the order in which they arrived.

For example, suppose the main communication line can carry only two packets at a time, but four computers start sending packets simultaneously. In this case, the first two packets are transmitted immediately, while the remaining packets wait in the router's queue; as space becomes available on the line, the queued packets are forwarded in turn. Packet switching is generally more efficient than circuit switching because no dedicated path is permanently reserved, bandwidth is shared dynamically among all users, network resources are used only when data is actually being transmitted, and any unused capacity can immediately be utilized by other users rather than sitting idle.

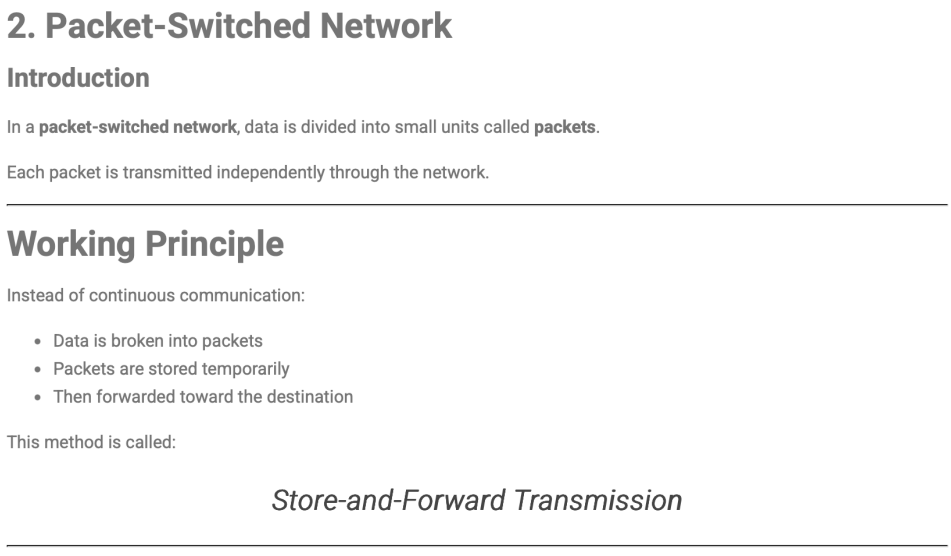


Figure: Packet-switched network — computers as end devices and routers acting as switches with queues for temporary storage of packets.

Figure 1.7 *A packet-switched network*

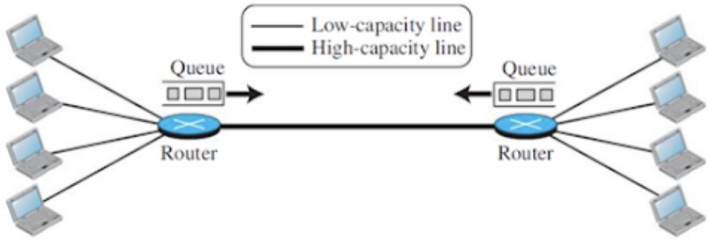


Figure: Packet switching under heavy traffic — packets queued at a router when the line reaches maximum capacity.

- Extra point: Circuit switching guarantees a fixed bandwidth and consistent delay for the duration of a call, which is why it was traditionally preferred for voice telephony.
- Extra point: Packet switching is the foundation of the modern Internet (via IP), because it allows efficient, statistical sharing of network links among many simultaneous users and applications.
- Extra point: A third approach, message switching (store-and-forward of entire messages), historically preceded packet switching but is rarely used today due to high delay and storage requirements.

1.7 Introduction to the Internet and Its Structure

An internet (with a lowercase "i") is a collection of two or more interconnected networks that can communicate with each other. The Internet (with a capital "I") is the world's largest and most well-known internetwork — a global network of interconnected computer networks that enables communication, information exchange, and resource sharing among millions of users worldwide.

The Internet is organized hierarchically into three levels: Backbone Networks, Provider Networks, and Customer Networks. Backbone networks are the largest and fastest networks, owned by major telecommunications companies such as AT&T, Verizon, Sprint, and NTT; they form the very core of the Internet and are interconnected with one another through peering points. Provider networks are smaller networks that connect to the backbone networks, pay fees to use backbone services, and in turn provide Internet access to their own customers; these are also called National or Regional ISPs. Customer networks are the networks used by homes, schools, colleges, offices, and businesses; they sit at the edge of the Internet and pay provider networks for their Internet service.

Figure 1.8 *The Internet today*

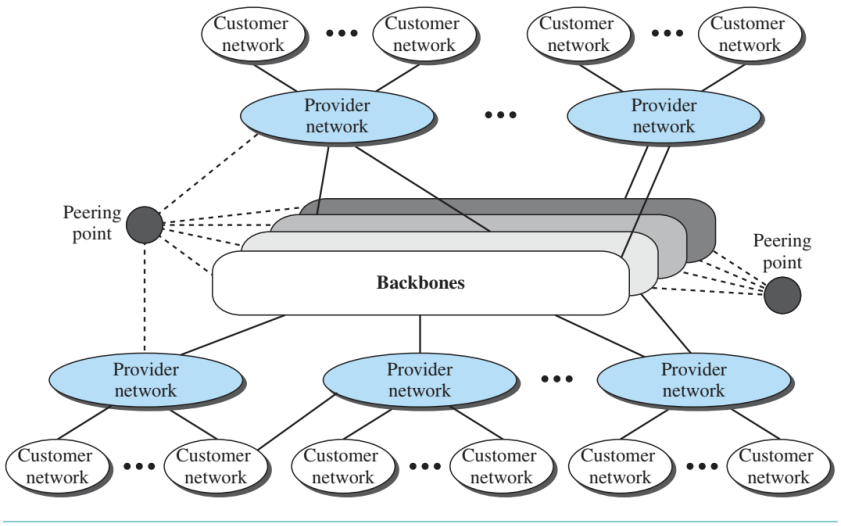


Figure: The three-level hierarchical structure of the Internet — backbone, provider, and customer networks.

Internet Service Providers (ISPs)

Both backbone networks and provider networks are collectively referred to as Internet Service Providers (ISPs). International ISPs correspond to the backbone networks, while National and Regional ISPs correspond to the provider networks that connect customers to those backbones.

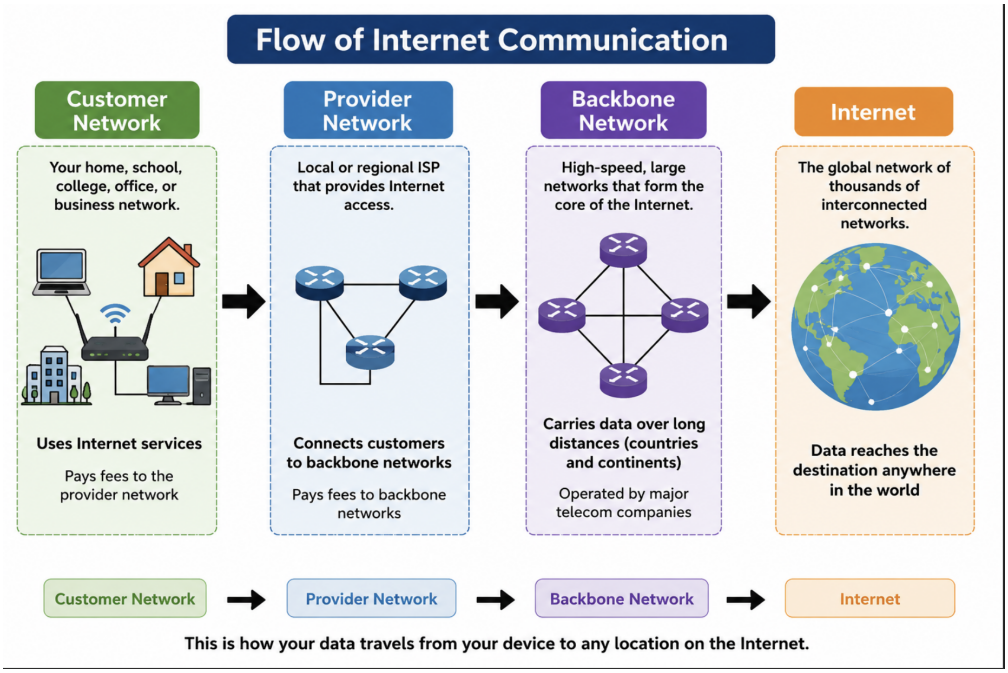


Figure: Classification of ISPs — International (backbone) versus National/Regional (provider) ISPs.

Accessing the Internet

To access the Internet, a user must first be connected to an Internet Service Provider through one of several available methods.

Using telephone networks, a dial-up service uses a modem to convert digital data into analog voice-band signals and connects to the ISP through an ordinary telephone line; this method is very slow, and the telephone line cannot be used for voice calls and Internet access at the same time. Digital Subscriber Line (DSL) technology, in contrast, uses the same existing telephone lines but provides much higher Internet speed, and it allows telephone and Internet services to be used simultaneously because it uses a different frequency band from voice calls.

Using cable networks, Internet access is provided over the same cable TV infrastructure used for television broadcasts, offering higher speeds than dial-up and DSL, although the actual speed experienced by a user may vary depending on how many other users are sharing the same cable segment. Using wireless networks, users connect to the Internet using wireless technologies, a method well suited to homes and small businesses and increasingly popular due to the mobility and convenience it offers. Finally, large organizations, universities, and corporations often use a direct connection to the Internet, in which they lease a high-speed WAN connection and connect directly to a regional ISP; such organizations can, in turn, act as a local ISP for their own internal networks and departments.

- Extra point: Fiber-to-the-Home (FTTH) has become increasingly common as an access technology, offering symmetrical high-speed connections that greatly exceed the capabilities of DSL or cable.
- Extra point: Mobile broadband technologies such as 4G LTE and 5G now provide an important alternative means of Internet access, especially in areas without fixed-line infrastructure.

1.8 The TCP/IP Protocol Suite

The TCP/IP Protocol Suite is the fundamental set of communication protocols used for data transmission over the Internet and most modern computer networks. The name "TCP/IP" comes from its two most important protocols: the Transmission Control Protocol (TCP), which ensures reliable and error-free delivery of data between devices, and the Internet Protocol (IP), which provides addressing and routing mechanisms to deliver data packets across networks. TCP/IP actually consists of a large collection of protocols organized into multiple layers, where each layer performs specific communication functions and works cooperatively with the other layers to enable seamless end-to-end data exchange.

Formally, the TCP/IP Protocol Suite can be defined as a hierarchical collection of communication protocols organized into layers that together facilitate reliable and efficient communication between

Why TCP/IP Uses Layering

Network communication is a complex process that involves data transmission, routing, error checking, and the provision of application-level services all at once. To manage this complexity, TCP/IP adopts a layered architecture in which the overall task of communication is divided into a set of smaller, more manageable sub-tasks, each handled by a distinct layer.

The advantages of this layered approach are numerous: each layer performs a specific and well-defined function; layers operate largely independently of one another, so changes inside one layer do not require changes in the others as long as the interface between layers remains the same; network design, implementation, and troubleshooting become considerably easier because problems can be isolated to a specific layer; and standardization at each layer enables interoperability among devices and technologies built by different manufacturers.

Each layer provides services to the layer directly above it and, in turn, receives services from the layer directly below it. This structured, layer-by-layer approach ensures efficient, reliable, and well-organized communication across the network. In simple terms, the TCP/IP model divides a complex communication process into smaller, manageable layers, making Internet communication both efficient and reliable.

OSI Model vs TCP/IP Model

The Open Systems Interconnection (OSI) model is a theoretical, seven-layer reference model consisting of the Application, Presentation, Session, Transport, Network, Data Link, and Physical layers. The TCP/IP model, which is the model actually implemented on the Internet, has only five layers: Application, Transport, Network, Data Link, and Physical. In the TCP/IP model, the functions of the OSI Application, Presentation, and Session layers are all combined into a single Application layer.

The reason TCP/IP does not have separate Session and Presentation layers is that some functions traditionally assigned to the OSI Session layer are instead handled by Transport layer protocols in TCP/IP; application programs are free to implement any needed Presentation or Session-layer functionality themselves if required; and combining these layers into a single Application layer simplifies the overall TCP/IP architecture without any real loss of functionality in practice.

1. **Application Layer (Layer 7)**
   - Provides services directly to users and applications.
   - Examples: HTTP, FTP, SMTP, DNS
2. **Presentation Layer (Layer 6)**
   - Handles data translation, encryption, and compression.
   - Ensures data is presented in a usable format.
3. **Session Layer (Layer 5)**
   - Establishes, manages, and terminates communication sessions between applications.
4. **Transport Layer (Layer 4)**
   - Provides end-to-end communication and reliability.
   - Examples: TCP, UDP
5. **Network Layer (Layer 3)**
   - Handles logical addressing and routing.
   - Example: IP
6. **Data Link Layer (Layer 2)**
   - Provides node-to-node communication.
   - Performs framing and error detection.
7. **Physical Layer (Layer 1)**
   - Transmits raw bits through the communication medium.

Figure: Mapping between the seven-layer OSI model and the five-layer TCP/IP model.

OSI vs TCP/IP

Similarities

1. Both are layered models.
2. Both support network communication.
3. Both divide communication into separate functions.

Differences

OSI Model	TCP/IP Model
7 Layers	5 Layers
Reference model	Protocol suite
Developed by ISO	Developed by DARPA/DoD
Presentation and Session layers are separate	Presentation and Session functions are included in the Application layer
Mainly used for learning and design	Used in the Internet and real-world networks

Figure: Comparison of OSI and TCP/IP layer functions.

TCP/IP Layers: Duties and Example Protocols

Summary of TCP/IP Layers

Layer	Main Duty	Examples of Protocols
Application Layer	Provides communication between application programs (process-to-process communication); allows users to access network services such as web browsing, email, and file transfer.	HTTP, FTP, SMTP, DNS, Telnet, SSH, SNMP
Transport Layer	Provides end-to-end communication between applications; performs segmentation, reliability, flow control, error control, and congestion control.	TCP, UDP, SCTP
Network Layer	Provides host-to-host communication and routes packets across multiple networks; handles logical addressing and routing.	IP, ICMP, IGMP, DHCP
Data Link Layer	Transfers packets across a single network link; performs framing, physical addressing, and error detection.	Ethernet, Wi-Fi, PPP
Physical Layer	Transmits raw bits as electrical, optical, or wireless signals through the transmission medium.	Fiber Optics, Twisted Pair Cable, Wireless Technologies

Logical Communication and Identical Objects Across Layers

Although data physically travels down through the layers of the sender, across the network, and then up through the layers of the receiver, it is conceptually useful to imagine that each layer on the sending host communicates directly ("logically") with the corresponding layer on the receiving host. This is illustrated by the concept of "identical objects" exchanged at each layer.

At the Application layer, the sender creates a message, and logically the Application layer at the sender communicates with the Application layer at the destination host; the object exchanged at this layer is simply called a Message. At the Transport layer, the message is divided into segments (if TCP is used) or user datagrams (if UDP is used), and logical communication occurs between the Transport layers of the source and destination hosts; the object exchanged here is called a Segment or User Datagram. At the Network layer, the Transport layer passes its data down to the Network layer, which adds an IP header, creating a Datagram (also called a Packet); intermediate routers process data only up to this layer, and the datagram remains logically the same as it travels across the Internet.

At the Data Link layer, the Network layer passes the datagram down to the Data Link layer, which encapsulates it into a Frame; unlike datagrams, frames are exchanged only between directly connected devices, and a new frame may be created at every hop along the path. Finally, at the Physical layer, frames are converted into a stream of Bits, which are then transmitted as electrical, optical, or wireless signals through the transmission medium.

TCP/IP Layered Architecture

The **TCP/IP Protocol Suite** follows a layered architecture to simplify network communication. Originally, TCP/IP was designed as a **four-layer model**, but it is commonly represented today as a **five-layer model** by separating the Data Link and Physical layers.

Five Layers of TCP/IP

Layer	Function
Application Layer	Provides network services directly to user applications and enables interaction between software and the network.
Transport Layer	Ensures end-to-end communication, data reliability, flow control, and error recovery between hosts.
Network Layer	Handles logical addressing, routing, and delivery of packets from source to destination across interconnected networks.
Data Link Layer	Provides node-to-node communication, framing, error detection, and access to the physical medium.
Physical Layer	Transmits raw bits over the communication medium using electrical, optical, or wireless signals.

Figure: Logical (peer-to-peer) communication between corresponding layers of the sender and receiver in the TCP/IP model.

Layer	Function	Responsibilities	Protocols / Devices	Examples
Application Layer	Provides network services directly to user applications.	User communication, data formatting, network services, resource sharing.	HTTP, HTTPS, FTP, SMTP, DNS	Web browsing, Email, File transfer, Domain name lookup
Transport Layer	Provides end-to-end process-to-process communication between hosts.	Segmentation, error control, flow control, reliable delivery, multiplexing.	TCP, UDP	File transfer (TCP), Web browsing (TCP), Video streaming (UDP), Online gaming (UDP)
Network Layer	Handles routing and delivery of packets across networks.	Logical addressing, routing, packet forwarding, path determination.	IP (IPv4/IPv6), Routers	Sending data between different networks, Internet communication
Data Link Layer	Provides communication between devices on the same network link.	Framing, physical addressing (MAC), error detection, media access control.	Ethernet, Wi-Fi, Switches	Communication within a LAN, Frame transmission between devices
Physical Layer	Transmits raw bits through the transmission medium.	Signal transmission, bit representation, cable and connector specifications.	Cables, Hubs, Repeaters, Fiber Optics	Electrical signals through cables, Wireless radio signals, Optical transmission through fiber

Figure: The objects exchanged at each layer — Message, Segment/Datagram, Packet, Frame, and Bits.

Communication in TCP/IP

Communication Process

Consider a scenario where **Computer A** sends data to **Computer B** through a switch, a router, and another switch.

- At the sender (Computer A), data passes through all five TCP/IP layers from top to bottom.
- The router processes the data at the Physical, Data Link, and Network layers.
- The switches process the data at the Physical and Data Link layers.
- At the receiver (Computer B), data passes through all five layers from bottom to top.

Role of Devices in TCP/IP

Device	Layers Used	Function
Host Computer	Application, Transport, Network, Data Link, Physical	Generates and receives data
Router	Network, Data Link, Physical	Routes packets between networks
Switch	Data Link, Physical	Forwards frames within a LAN

Figure: Data flow through the TCP/IP layers from the Application layer down to the Physical layer.

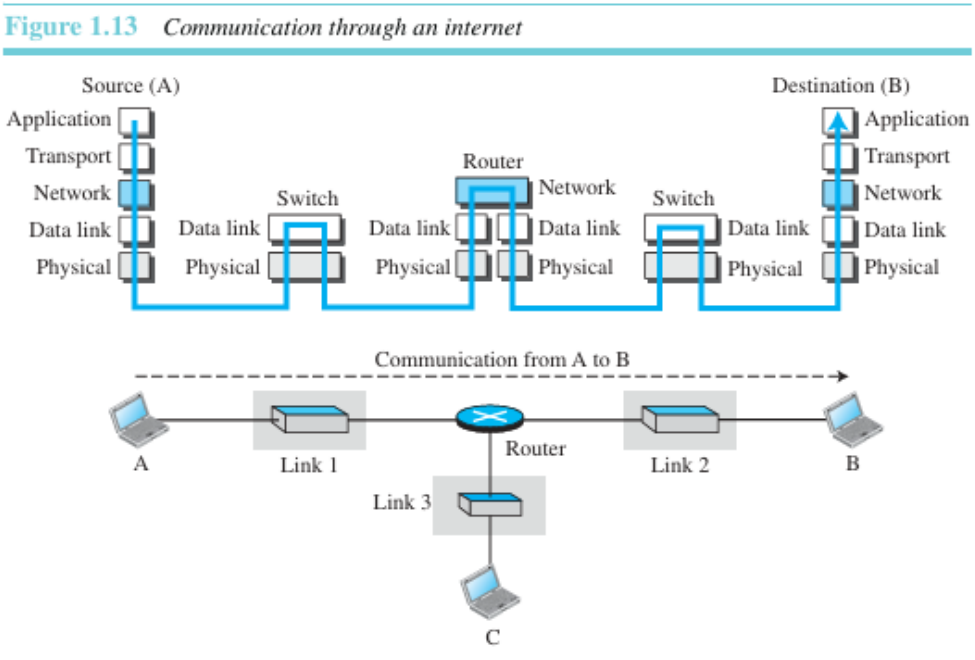


Figure: Router processing shown only up to the Network layer, illustrating why the datagram remains unchanged end-to-end.

End-to-End and Hop-to-Hop Communication

Communication Type	Layers Involved	Description
End-to-End Communication	Application, Transport, Network	Communication between source and destination hosts across the Internet.
Hop-to-Hop Communication	Data Link, Physical	Communication between two directly connected devices.

End-to-End Communication

The upper three layers provide communication from the source host to the destination host:

- Application Layer
- Transport Layer
- Network Layer

Hop-to-Hop Communication

The lower two layers provide communication between neighboring devices:

- Data Link Layer
- Physical Layer

Figure: Frame creation at each hop between directly connected devices in the Data Link layer.

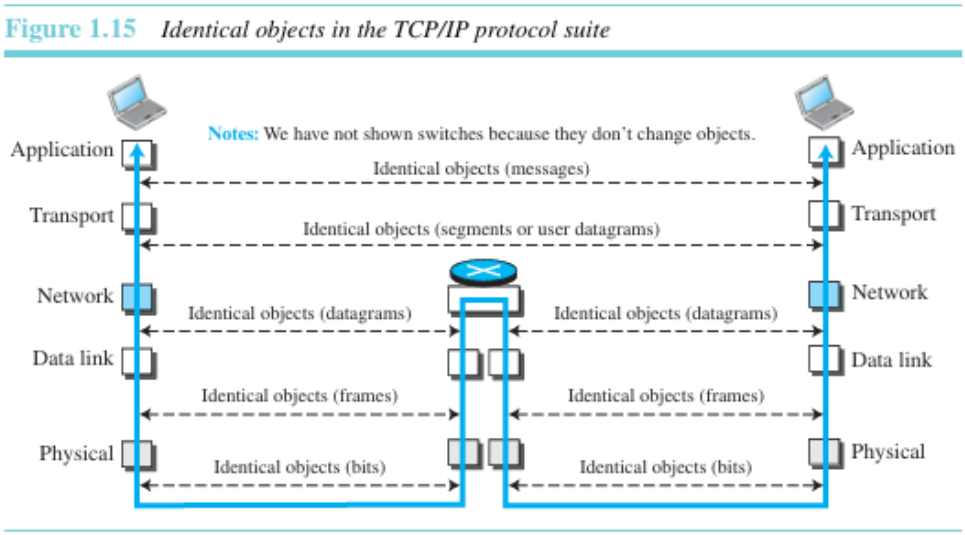


Figure: Conversion of frames into a stream of bits for transmission at the Physical layer.

Encapsulation and Decapsulation

Encapsulation is the process by which each layer adds its own header (and sometimes trailer) information to the data received from the layer above it, as the data moves down the protocol stack at the sending host. At the source host, the Application layer first creates the data, called a Message. The Transport layer then receives this message and adds a Transport header, creating a Segment (in the twork layer adds an IP header to this

segment, creating a Datagram (or Packet). The Data Link layer adds a Frame header (and often a trailer), creating a Frame. Finally, the Physical layer converts the frame into a stream of Bits and transmits them through the communication medium.

This overall encapsulation flow can be summarized as: Message → Segment/User Datagram → Datagram → Frame → Bits. Decapsulation is simply the reverse process that occurs at the receiving host: as the bits travel up through the layers, each layer strips off the header added by its counterpart layer at the sender, ultimately recovering the original message at the Application layer.

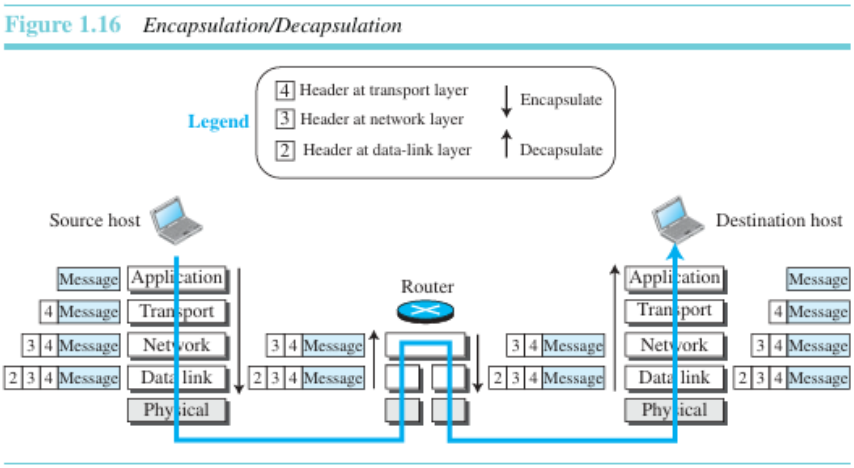


Figure: Encapsulation and decapsulation of data as it moves down and up the TCP/IP protocol stack.

Decapsulation and Encapsulation at a Router

1. The router receives the frame and removes the **Data Link Header** (Decapsulation).
2. The router examines the **Destination IP Address** in the datagram.
3. The router determines the next hop using its routing table.
4. The datagram is then passed to the outgoing Data Link Layer.
5. A new frame is created by adding a new Data Link Header (Encapsulation).
6. The frame is transmitted to the next device.

Decapsulation at the Destination Host

1. The **Physical Layer** receives the bits.
2. The **Data Link Layer** removes the frame header and extracts the datagram.
3. The **Network Layer** removes the IP header and extracts the segment.
4. The **Transport Layer** removes the transport header and extracts the message.
5. The **Application Layer** receives the original message.

Decapsulation Flow

Bits → Frame → Datagram → Segment/User Datagram → Message

ort, Network, and Data Link headers.

Addressing in TCP/IP

Communication between two devices requires a **source address** and a **destination address.** In the TCP/IP model, different layers use different types of addresses.

Types of Addresses in TCP/IP

Layer	Address Type	Purpose	Packet Name
Application Layer	Names	Identify users or services	Message
Transport Layer	Port Numbers	Identify application processes	Segment / User Datagram
Network Layer	Logical Addresses (IP Address)	Identify hosts on the Internet	Datagram
Data Link Layer	Link-Layer (MAC) Addresses	Identify devices on a local network	Frame
Physical Layer	No Address	Transmits bits	Bits

Figure: Encapsulation flow — Message to Segment/Datagram to Frame to Bits.

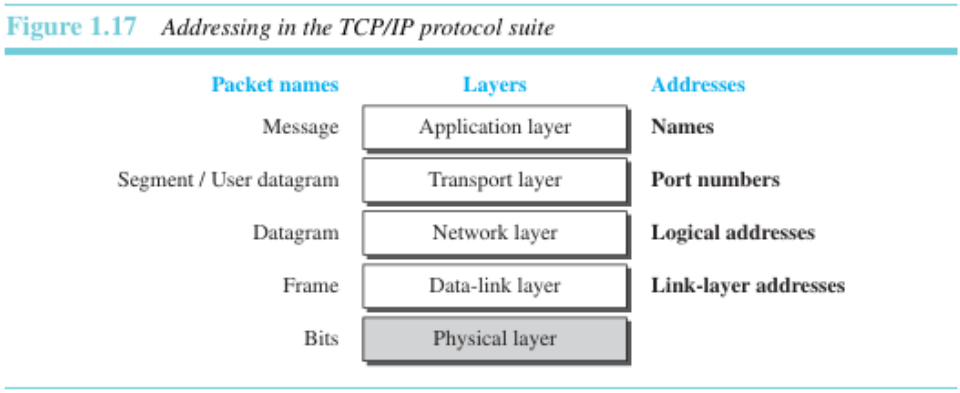


Figure: Decapsulation at the receiving host, where each layer removes the corresponding header.

Examples

- **Application Layer:** www.google.com ↗, user@gmail.com ↗
- **Transport Layer:** Port 80 (HTTP), Port 25 (SMTP)
- **Network Layer:** 192.168.1.10, 172.16.0.5
- **Data Link Layer:** 00:1A:2B:3C:4D:5E (MAC Address)

Figure: Overall encapsulation/decapsulation process shown end-to-end between sender and receiver.

Multiplexing and Demultiplexing

Multiplexing occurs at the source (sender) side of a communication. A protocol operating at a particular layer can receive data from multiple different higher-layer protocols simultaneously; it encapsulates this data with the appropriate header and forwards it down to the lower layer. A special

header field is used to identify which protocol originally generated the data, so that it can later be delivered to the correct protocol at the destination.

For example, at the Transport layer, TCP can receive data originating from application-layer protocols such as HTTP, FTP, SMTP, or DNS, while UDP can receive data from protocols such as DNS, DHCP, or VoIP applications. At the Network layer, IP can receive TCP segments, UDP user datagrams, ICMP packets, or IGMP packets, all of which it multiplexes into IP datagrams. At the Data Link layer, a single frame can carry IP datagrams, ARP packets, or other network-layer protocol data.

Demultiplexing occurs at the destination (receiver) side and is the reverse of multiplexing. A protocol at the receiver examines the header information of the received packet, and based on the protocol identifier found there, delivers the packet to the correct higher-layer protocol before decapsulating it and passing it further upward. For example, at the Data Link layer, the receiver delivers the payload to either IP or ARP based on the frame's type field; at the Network layer, packets are delivered to TCP, UDP, ICMP, or IGMP based on the protocol field in the IP header; and at the Transport layer, data is delivered to the correct application, such as HTTP, FTP, SMTP, or DNS, using port numbers.

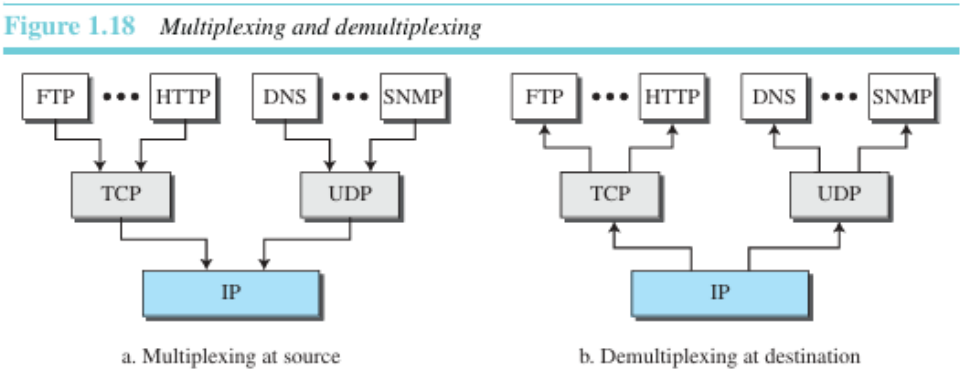


Figure: Multiplexing at the Transport and Network layers — multiple protocols feeding into a single lower-layer protocol.

OSI Model

Introduction

1. The **OSI (Open Systems Interconnection) Model** was developed by the **International Organization for Standardization (ISO)** in the late 1970s.
2. ISO is an international organization that develops worldwide standards.
3. OSI is a **reference model**, not a protocol.
4. It provides a framework for designing and understanding network communication.
5. The main goal of the OSI model is to enable communication between different computer systems regardless of their hardware and software architecture.
6. The OSI model uses a **layered architecture** with seven layers.

Definition

- The **OSI Model** is a seven-layer reference model that standardizes network communication and enables interoperability between different computer systems.

Figure: Demultiplexing at the receiver — a packet delivered to the correct higher-layer protocol using a protocol/port identifier.

Figure 1.19 *The OSI model*

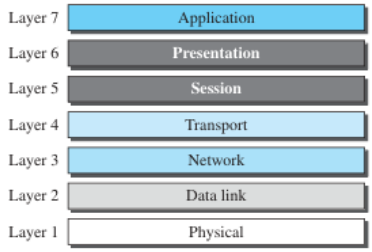


Figure 1.20 *TCP/IP and OSI model*

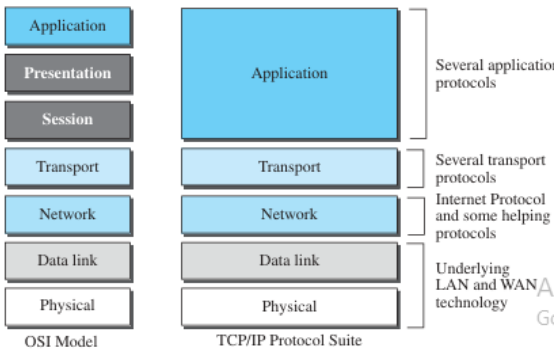


Figure: Multiplexing/demultiplexing summary across the TCP/IP layers.

1. **Application Layer (Layer 7)**
   - Provides services directly to users and applications.
   - Examples: HTTP, FTP, SMTP, DNS
2. **Presentation Layer (Layer 6)**
   - Handles data translation, encryption, and compression.
   - Ensures data is presented in a usable format.
3. **Session Layer (Layer 5)**
   - Establishes, manages, and terminates communication sessions between applications.
4. **Transport Layer (Layer 4)**
   - Provides end-to-end communication and reliability.
   - Examples: TCP, UDP
5. **Network Layer (Layer 3)**
   - Handles logical addressing and routing.
   - Example: IP
6. **Data Link Layer (Layer 2)**
   - Provides node-to-node communication.
   - Performs framing and error detection.
7. **Physical Layer (Layer 1)**
   - Transmits raw bits through the communication medium.

Figure: Port numbers used to demultiplex data to the correct application at the Transport layer.

- Extra point: Multiplexing/demultiplexing at the Transport layer is achieved specifically through the use of 16-bit port numbers, which uniquely identify an application process on a host.
- Extra point: Well-known port numbers (0–1023) are reserved for standard services — for example, HTTP uses port 80, HTTPS uses port 443, and FTP uses ports 20 and 21.

1.9 Application Layer

The Application layer is the topmost layer of the TCP/IP protocol suite. It provides services directly to users and application programs and enables communication between applications running on different hosts. Communication at this layer is achieved through a logical connection between the Application layers on different hosts — the two Application layers behave as if a direct connection exists between them, even though, in reality, data passes through all of the lower layers before actually reaching the destination. Communication at the Application layer is therefore best described as process-to-process communication, in which a process on one host sends a request to a process on another host and receives a response in return.

This is called logical communication because the connection between the two Application layers is imaginary — actual data transfer physically occurs through the Transport, Network, Data Link, and Physical layers. For example, when a user opens a website, the web browser sends an HTTP request, which travels through all of the TCP/IP layers before the destination web server receives it; the server

then sends an HTTP response back to the browser along the same layered path. To the two applications involved, it appears as though they are communicating directly with one another.

Application-Layer Paradigms

Internet communication fundamentally requires two application programs, running on two different computers, to exchange messages with each other through the Internet. The relationship between these two communicating programs is defined by what is called an Application-Layer Paradigm. Depending on the paradigm, an application program may request a service, provide a service, or perform both functions simultaneously. Two major paradigms have evolved for application-layer communication: the Client-Server paradigm and the Peer-to-Peer (P2P) paradigm.

Client–Server Paradigm

The Client–Server paradigm is a communication model in which a server process provides services and a client process requests those services through the Internet. In this model, the server is the service provider, while the client requests services from the server. The server runs continuously and waits for client requests, whereas the client starts only when a service is actually needed. Many clients can connect to a single server at the same time, but the roles of client and server are fundamentally different and cannot be interchanged; separate client and server programs are typically required for each distinct service offered.

The working of the Client–Server model can be summarized as follows: the client initiates a connection to the server; the client sends a service request; the server processes the request; the server sends a response back to the client; and the client receives the requested service.

A real-life analogy is a telephone directory service, in which the directory center acts as the server and the subscriber acts as the client — the directory center is always available, while the subscriber contacts it only when specific information is needed.

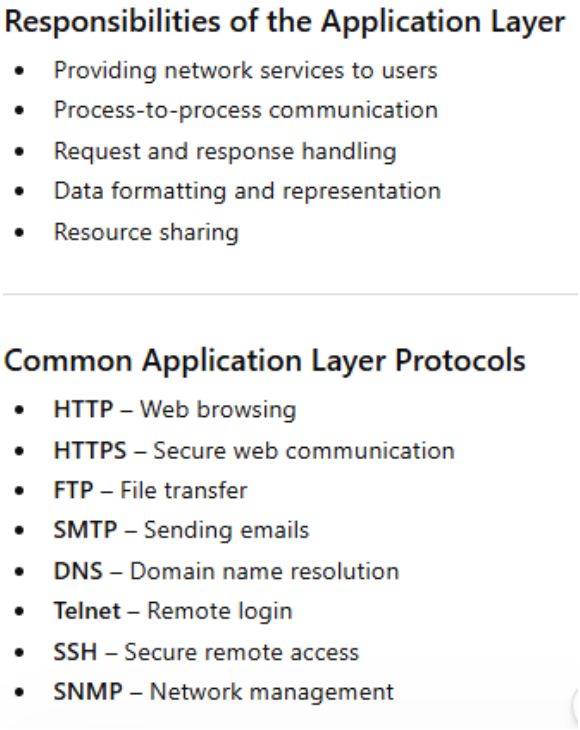


Figure: Client-Server communication model — clients requesting services from a continuously running server.

The advantages of the Client-Server paradigm include centralized control and management, easy maintenance and updates, efficient resource sharing, and better security and data management, since all critical data and logic reside on a controlled server. The disadvantages include a heavy workload placed on the server, the requirement for powerful and often expensive server hardware, the fact that server failure affects all connected clients simultaneously, high setup and maintenance costs, and the possibility of degraded performance when a very large number of clients connect at the same time. Common examples of client-server applications include the World Wide Web (WWW) using HTTP, the File Transfer Protocol (FTP), Secure Shell (SSH), and e-mail services.

Peer-to-Peer (P2P) Paradigm

The Peer-to-Peer (P2P) paradigm is a communication model in which every computer, called a peer, can act as both a service provider and a service requester, eliminating the need for a dedicated server that must run continuously. In this model, no dedicated server is required; responsibility for providing services is shared among all peers; a given peer can provide services to other peers, receive services from other peers, or do both simultaneously; each computer has equal status within the network; and the overall workload is distributed among multiple peers rather than being concentrated on one machine.

The working of the P2P model is as follows: a peer requests a service from another peer; the receiving peer provides the requested service; and the roles of the two peers can change at any time, since the same peer can act as both a client and a server depending on the situation.

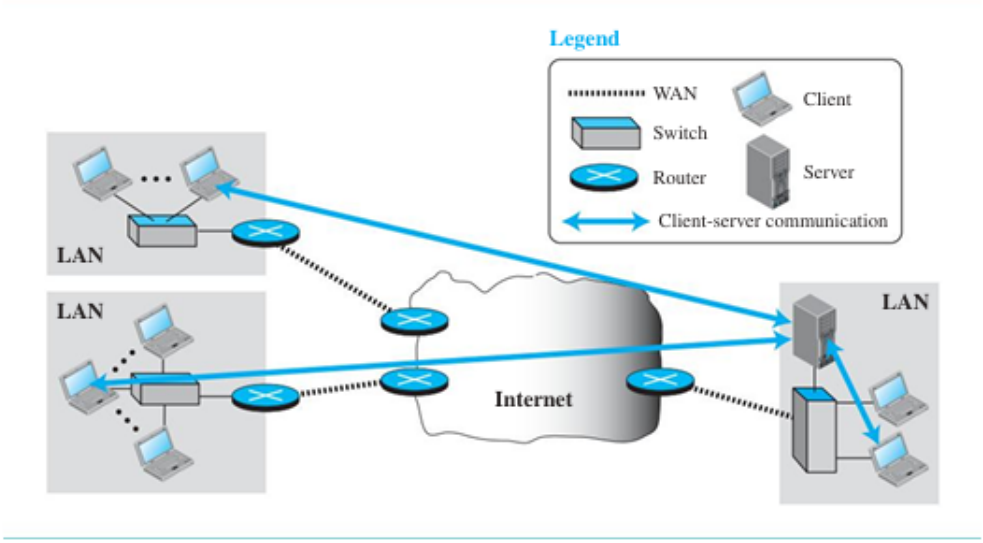


Figure: Peer-to-Peer (P2P) communication model — every peer can act as both client and server.

The advantages of the P2P paradigm include the elimination of expensive dedicated servers, making it cost-effective; easy scalability as new peers join; distribution of workload among peers; and better overall utilization of network resources. The disadvantages include the fact that security becomes more challenging to enforce in a fully decentralized system, distributed services are more difficult to manage, data availability depends on which peers happen to be online at a given time, and the model is not suitable for every type of Internet application. Common applications of the P2P paradigm include Internet telephony, file sharing, BitTorrent, IPTV, and voice and video communication. In Internet telephony, for instance, both users communicate directly with each other, and no dedicated server is required to continuously provide the calling service; similarly, in file sharing, users exchange files directly, and no permanent server needs to be maintained by any single user.

Figure 2.3 *Example of a peer-to-peer paradigm*

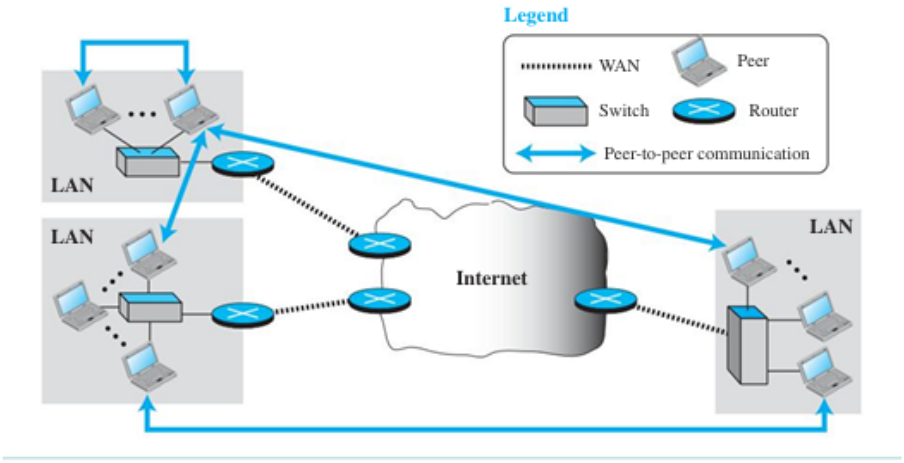


Figure: Examples of P2P applications — file sharing and Internet telephony between peers.

Mixed Paradigm

The Mixed paradigm combines features of both the Client-Server and Peer-to-Peer paradigms in order to take advantage of the strengths of each model. In this approach, a central server is used only for specific, limited tasks such as locating peers, while the actual service or data transfer occurs directly between the peers themselves. This design reduces the load placed on the server while still improving scalability and efficiency compared to a pure client-server system.

The working of the Mixed paradigm proceeds as follows: a client first contacts a server to find the network address of a suitable peer; the server provides this peer's address to the client; the client then establishes a direct connection with that peer; data or services are exchanged directly between the two peers; and the server is no longer involved once the actual communication begins. The advantages of this approach include reduced server workload, greater scalability than a pure client-server architecture, cost-effectiveness, efficient resource sharing, and the ability to combine centralized control with distributed communication. Applications of the Mixed paradigm include peer-to-peer file sharing, Internet telephony services such as Skype, BitTorrent, and IPTV.

Sockets, Socket Addresses, and the API

The Client-Server paradigm requires additional supporting concepts to make communication between processes possible. An Application Programming Interface (API) is a set of instructions that allows an application program to communicate with the operating system and the underlying TCP/IP protocol suite. The main functions provided by an API include opening a connection, sending data, receiving data, and closing a connection. Common APIs used for this purpose include the Socket Interface, the Transport Layer Interface (TLI), and STREAMS.

A socket is an abstraction that acts as an endpoint for communication between a client process and a server process. A socket is created by an application program, and communication actually occurs between two sockets — one on the client side and one on the server side. The client sends its requests through its own socket, and the server sends its responses through its own socket; in this sense, a socket acts as both a source and a destination for data.

A Socket Address is used to uniquely identify a process running on a particular computer. It consists of two components: a 32-bit IP Address, which identifies the specific computer on the network, and a 16-bit Port Number, which identifies the specific application process running on that computer. This relationship is often summarized with the formula: Socket Address = IP Address + Port Number.

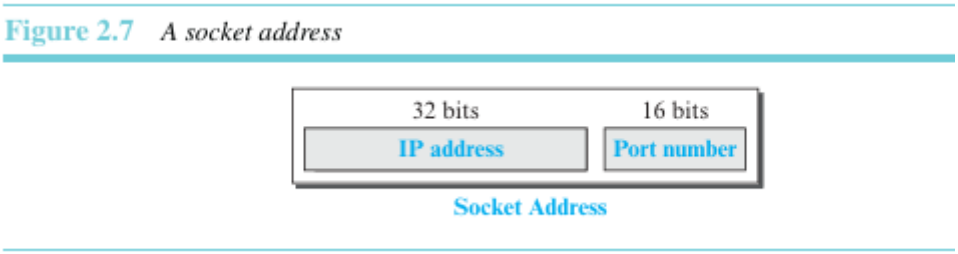


Figure: A socket as the communication endpoint between a client process and a server process.

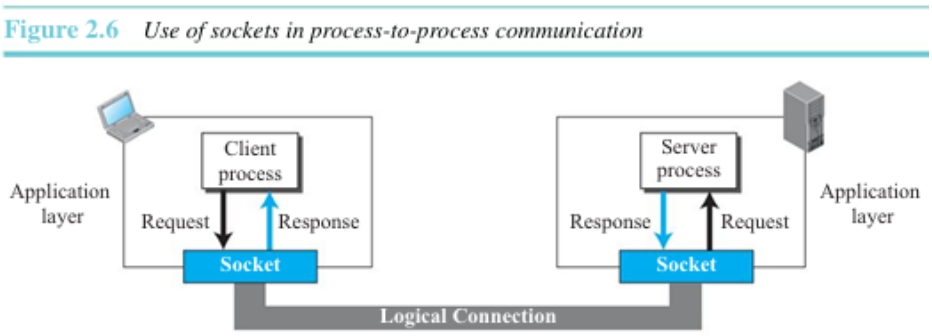


Figure: Composition of a socket address — IP address combined with a port number.

On the server side, the local socket address has an IP address known to the operating system and a port number that is fixed and assigned to the server (for example, HTTP conventionally uses port 80), while the remote socket address is obtained from the client's request and changes with every new client connection. On the client side, the local socket address has an IP address assigned by the operating system and a temporary (ephemeral) port number assigned for that particular communication session, while the remote socket address consists of the server's IP address and port number, which can be obtained directly or resolved through DNS.

Using Services of the Transport Layer

Application processes rely on the services provided by the Transport layer to actually carry out communication over the network. The TCP/IP protocol suite provides three commonly used transport-layer protocols: UDP (User Datagram Protocol), TCP (Transmission Control Protocol), and SCTP (Stream Control Transmission Protocol). The particular transport protocol chosen for an application affects both its performance characteristics and the capabilities available to it.

UDP (User Datagram Protocol) is a connectionless, unreliable, message-oriented protocol that is fast and simple, and it provides no flow control or congestion control of its own. Its advantages include low overhead and faster transmission, making it suitable for small messages; typical applications include multimedia streaming, network management, and VoIP applications. A useful analogy for UDP is sending letters through the post office, where each message (letter) is treated independently of the others.

TCP (Transmission Control Protocol) is a connection-oriented, reliable, byte-stream-oriented protocol that provides flow control, error control, and congestion control. Its advantages include reliable data delivery, since lost or corrupted data can be automatically retransmitted, making it suitable for large data transfers. Typical applications include web browsing (HTTP/HTTPS), file transfer (FTP), and e-mail services. TCP can be compared to a telephone conversation, in which a connection is established before any information is actually exchanged.

SCTP (Stream Control Transmission Protocol) is a connection-oriented, reliable, message-oriented protocol that supports multistream communication and can maintain a communication session even if one network path fails. Its advantages include combining desirable features of both TCP and UDP, being reliable and message-oriented at the same time, and offering better fault tolerance. Typical applications include multimedia communication, telecommunication systems, and other applications that require particularly high reliability.

1.10 Standard Client-Server Applications

During the evolution of the Internet, several standard client-server applications have been developed to provide different essential services. Understanding these applications is important because it also helps in designing new, customized network applications. The major standard applications include HTTP/WWW for accessing web pages and websites, FTP for transferring files between computers, E-mail for sending and receiving electronic messages, TELNET for remote login to another computer, SSH for secure remote access and login, and DNS for mapping domain names to IP addresses.

World Wide Web (WWW)

The World Wide Web (WWW) was proposed by Tim Berners-Lee in 1989 at CERN with the original goal of facilitating the sharing of research information among scientists. The commercial use of the

Web began in the early 1990s and has since grown into the dominant application of the Internet. The Web can be described as a global repository of information, in which documents called web pages are stored on servers located all around the world.

The popularity of the Web can be attributed to two especially important characteristics. Its distributed nature means that web pages are stored on many different servers worldwide, allowing the Web to grow without any practical limit. Its linked structure means that web pages can be connected to one another through hyperlinks, enabling easy navigation between related documents. The concept of linking documents together originated from the idea of Hypertext, which allows a user to move from one document to another simply by clicking a link. Today, the term Hypermedia is more commonly used, since modern web pages can contain not just text but also images, audio, video, and animations.

Architecture of the World Wide Web

The World Wide Web is fundamentally a distributed client-server system that allows users to access information over the Internet. In this architecture, the client is the user's computer or device, which uses a web browser such as Chrome, Firefox, or Edge to request and view web pages. A server stores web pages and delivers them to clients upon request. Information on the Web is distributed across many different locations, called sites, and each site typically contains one or more web pages.

A web page is a file identified by a unique name and address, known as a URL. Web pages can be classified as either simple web pages, which contain only information and no links to other pages, or composite web pages, which contain one or more hyperlinks to other web pages within the same site or on different sites. Hyperlinks connect web pages together and allow users to navigate freely from one page to another.

The overall working of the WWW architecture proceeds as follows: the user enters a URL into a web browser; the browser (acting as the client) sends a request to the appropriate web server; the server locates the requested web page; the server sends the page back to the browser; and the browser then displays the page to the user, including any hyperlinks that lead to other pages.

Retrieval of Web Documents

A web document may contain references, in the form of hyperlinks, to other files such as text documents, images, videos, or additional web pages. Consider, for example, a scientific document consisting of File A, the main document, stored at Site I; File B, a large image, also stored at Site I; and File C, a referenced text file stored at a different location, Site II.

To view the complete document with all of its referenced material, three separate transactions are typically needed. The first transaction retrieves the main document, File A, which itself contains references (links) to File B and File C. The second transaction is triggered only when the user actually

clicks the image reference, at which point File B (the image) is retrieved. The third transaction is triggered only when the user clicks the text-file reference, retrieving File C. Importantly, File A, File B, and File C are independent web pages or files, each with its own name and address (URL); even though File B and File C are referenced from within File A, they can be accessed independently, and retrieving one file does not automatically retrieve the others.

Web Client (Browser)

A web client, or browser, is the software application used to access and display web pages. The architecture of a typical browser consists of three main components. The Controller receives input from the user through the keyboard or mouse and controls overall access to web documents. Client Protocols are used to communicate with web servers and include protocols such as HTTP and FTP. Interpreters are responsible for interpreting and displaying the actual contents of web pages, and they include an HTML interpreter, a Java interpreter, and a JavaScript interpreter.

The overall working of a browser proceeds as follows: the user enters a URL or clicks on a link; the Controller sends a request using the appropriate client protocol; the requested document is retrieved from the server; an Interpreter processes the retrieved document; and the resulting document is displayed on the screen. Well-known examples of browsers include Internet Explorer, Netscape Navigator, Firefox, and Google Chrome.

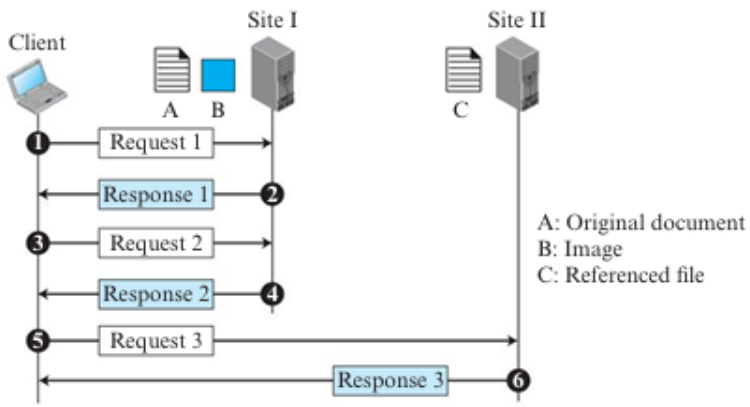


Figure: Internal architecture of a web browser — Controller, Client Protocols, and Interpreters.

Web Server

A web server is responsible for storing web pages and delivering them to clients whenever they are requested. Its main functions are to store web pages and other files, to receive requests from browsers, and to send the requested documents back to the requesting clients. Examples of widely used web servers include the Apache Server and Microsoft Internet Information Server (IIS).

To summarize this section: a web document may contain references to other files, which require separate transactions to retrieve; all referenced files are independent and have unique addresses; a web browser consists of a controller, client protocols, and interpreters that together retrieve and display web pages; and a web server stores web pages, responds to client requests, employs caching for faster access, and uses multithreading or multiprocessing to serve multiple users efficiently at the same time.

Figure 2.9 *Browser*

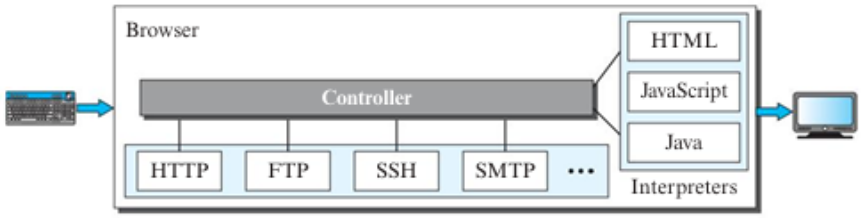


Figure: Interaction between a web client (browser) and a web server for retrieving a web page.

Uniform Resource Locator (URL)

A Uniform Resource Locator (URL) is the unique address used to identify and locate a web page or resource on the Internet. To access a web page, four pieces of information are generally required: the Protocol, the Host, the Port, and the Path.

The Protocol specifies the client-server application used to access the resource and indicates how data is transferred between the browser and the server; examples include HTTP, HTTPS, and FTP (for instance, "http://"). The Host identifies the server on which the resource is stored and can be given either as an IP address (such as 64.23.56.17) or as a domain name (such as forouzan.com). The Port is a 16-bit integer that identifies a specific service on the server; it is usually predefined for common protocols — for example, HTTP uses port 80 and HTTPS uses port 443 — and if the default port for a protocol is being used, it is usually omitted from the URL that is displayed to the user.

The Path specifies the exact location and name of the file on the server, consisting of one or more directories followed by the file name; for example, in the UNIX-style path "/top/next/last/myfile," "top" is the top-level directory, "next" and "last" are subdirectories, and "myfile" is the actual file name. Without a port number (the most common case), the URL format is protocol://host/path, as in "http://www.forouzan.com/books/index.html"; with an explicit port number, the format becomes protocol://host:port/path, as in "http://www.forouzan.com:80/books/index.html." In summary, a URL uniquely identifies a resource on the Internet and consists of a protocol (which specifies how the resource is accessed), a host (which identifies the server), a port (which identifies the service on that server), and a path (which identifies the exact file or resource).

Web Documents

Web documents can be classified into three main types. Static documents have fixed content that is stored on a web server; users can only view a copy of the document, and its content remains the same unless it is explicitly updated by a server administrator. Static documents are typically created using languages such as HTML, XML, XSL, and XHTML; a college website's general information page is a good example. Dynamic documents are created by the server at the moment they are requested by a browser, so their content may change for each individual request; they are generated using programs or scripts such as JSP, ASP, or ColdFusion, and a common example is a webpage that displays the current date and time. Active documents contain programs or scripts that run on the client (browser) side rather than on the server, and they are used for animations, interactive forms, and other forms of user interaction; they are typically created using Java Applets or JavaScript, with interactive games or animated graphics on a webpage serving as good examples.

HyperText Transfer Protocol (HTTP)

HTTP (HyperText Transfer Protocol) is the protocol used to transfer web pages between a web browser (the client) and a web server. In this exchange, the client sends an HTTP request, and the server responds with an HTTP response. HTTP uses TCP as its underlying transport protocol, with the server conventionally listening on port 80, while the client uses a temporary (ephemeral) port for the connection. The main features of HTTP are that it is a client-server protocol; it uses TCP for reliable communication; it requires a TCP connection to be established before any data can be transferred; and the connection is typically terminated once communication is complete.

Non-Persistent and Persistent Connections

When a web page contains multiple embedded objects, such as images or videos, HTTP can retrieve all of these objects using either a non-persistent connection or a persistent connection. In a non-persistent connection, a separate TCP connection is created for each individual request and response; this approach was used in HTTP/1.0. The steps involved are: the client opens a TCP connection and sends a request; the server sends back the response and then closes the connection; and the client receives the data and also closes its end of the connection. For example, if a webpage contains N images, one connection is required for the HTML page itself, and N additional connections are required for the N images, giving a total of N + 1 connections. The disadvantages of this approach include high overhead, more time spent on connection setup and termination, increased workload on the server, and the need for more buffers and other resources.

In a persistent connection, which is the default behavior in HTTP/1.1, a single TCP connection is used for multiple requests and responses instead of opening a new connection for every object. The advantages of persistent connections include reduced overhead, faster page loading, less overall

network traffic, and better server performance, since the cost of setting up and tearing down a connection is incurred only once rather than repeatedly.

Consider a specific nonpersistent-connection example in which a webpage contains a text file with one linked image, and both files are stored on the same server; in this case, two separate TCP connections are required — one for the text file and one for the image file. Each connection requires its own connection establishment (TCP handshake), data transfer, and connection termination. This illustrates that multiple objects require multiple connections when using a non-persistent approach, leading to increased round-trip time (RTT) due to repeated handshakes, the need for additional buffers and variables for each connection, and generally higher overhead, especially on the server side.

A persistent connection, by contrast, keeps a single TCP connection open across multiple requests and responses; the server keeps the connection open after sending a response, and the connection is only closed either at the client's request or when a timeout occurs. The sender usually specifies the length of the data being sent so the receiver knows when the response ends; however, for dynamically generated documents where the length may not be known in advance, the server simply closes the connection after transmission to signal the end of the data. Persistent connections save time and network resources, require only one set of buffers and variables at each site, eliminate the need for repeated connection establishment and termination, and ultimately reduce RTT and improve overall performance.

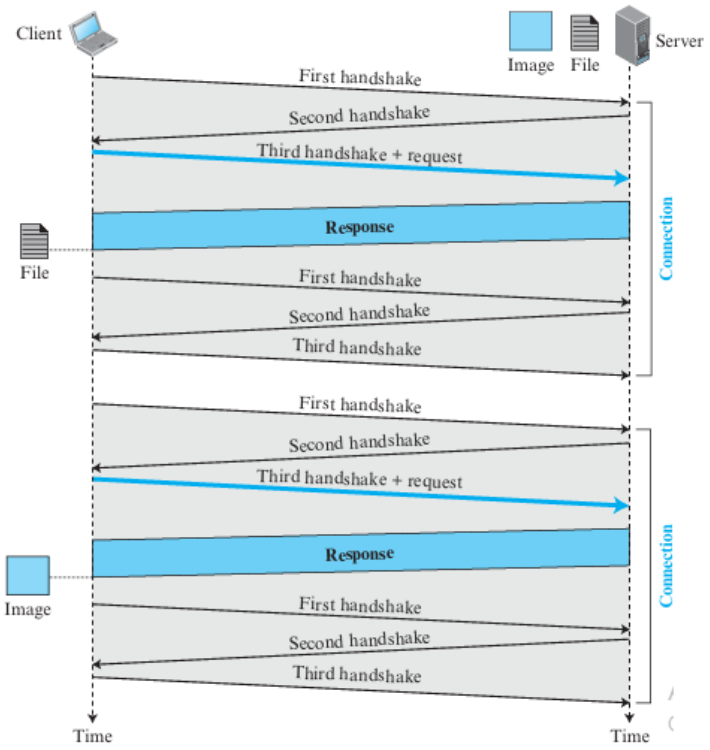


Figure: Comparison of non-persistent and persistent HTTP connections for retrieving a webpage with multiple objects.

HTTP Request Message

An HTTP request message consists of four parts: the Request Line, the Header Lines, a Blank Line, and an optional Body.

The Request Line contains three fields, separated by spaces and terminated by a CRLF (Carriage Return and Line Feed) sequence: the Method, which specifies the type of request being sent by the client; the URL, which specifies the address and name of the requested web page (for example, "/index.html") and identifies the exact resource to be accessed; and the Version, which specifies the HTTP protocol version being used, with "HTTP/1.1" being the most common current version.

Zero or more Request Header Lines may follow the request line, providing additional information from the client to the server and allowing the client to specify preferences and conditions for the request. Header lines follow the general format "Header-Name: Value"; common examples include "Host: www.example.com," "Accept: text/html," "User-Agent: Chrome," and "If-Modified-Since: Mon, 10 Jun 2025." The Blank Line separates the header section from the body section and marks the end of the headers. The Body, which is optional, is present only in some request messages and is most commonly used with PUT and POST methods; it may contain comments, form data, files to be uploaded, or other information used to modify a resource on the server.

Figure 2.12 *Formats of the request and response messages*

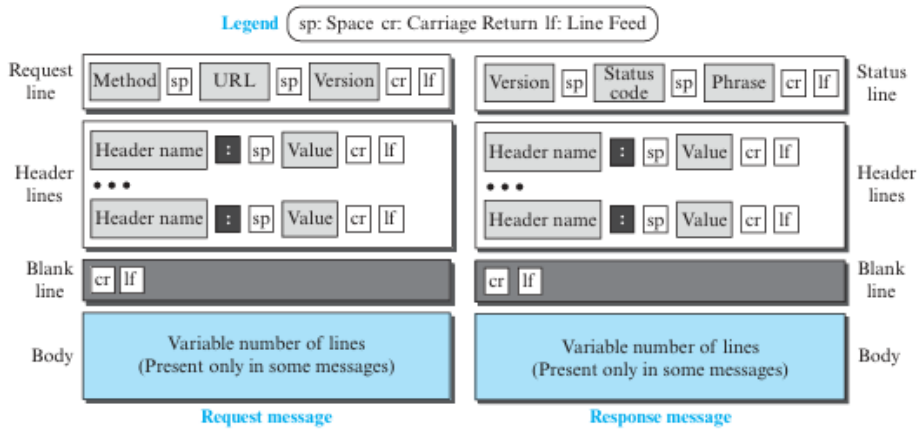


Figure: Structure of an HTTP request message — request line, header lines, blank line, and body.

HTTP Response Message

An HTTP response message is sent by the server back to the client and, like the request message, consists of four parts: the Status Line, the Header Lines, a Blank Line, and an optional Body.

The Status Line is the first line of a response message, following the format "Version SP Status-Code SP Status-Phrase CRLF." The Version field specifies the HTTP protocol version used by the server,

er that falls within a specific range

depending on its meaning: 100–199 indicates informational messages, 200–299 indicates a successful request, 300–399 indicates redirection to another URL, 400–499 indicates a client-side error, and 500–599 indicates a server-side error. Some commonly encountered status codes include 200 (OK), 301 (Moved Permanently), 302 (Found, or Temporary Redirect), 404 (Not Found), and 500 (Internal Server Error). The Status Phrase is simply a short textual explanation of the status code, such as "200 OK," "404 Not Found," or "500 Internal Server Error."

Zero or more Response Header Lines may follow the status line, providing additional information from the server to the client and describing either the returned document or the server itself, using the same "Header-Name: Value" format as request headers. As with the request message, a Blank Line separates the header section from the body section and marks the end of the headers. The Body, when present, contains the actual document or data requested by the client, which may include HTML pages, images, audio files, video files, or descriptive error messages.

Table 2.1 *Methods*

Method	*Action*
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

Figure: Structure of an HTTP response message — status line, header lines, blank line, and body.

File Transfer Protocol (FTP)

The File Transfer Protocol (FTP) is the standard TCP/IP protocol used for transferring files from one host to another. FTP was designed to solve problems that arise from different file naming conventions, data representations, and directory structures on different systems, and it is generally preferred over HTTP for transferring large files or files in a variety of formats.

The FTP architecture uses a client-server model with two separate TCP connections rather than just one. On the client side, the components include a User Interface, a Client Control Process, and a Client Data Transfer Process; on the server side, the components include a Server Control Process and a Server Data Transfer Process. The two connections used by FTP are the Control Connection, which transfers commands and responses, and the Data Connection, which transfers the actual file data. This separation of control information from data transfer is a key advantage of FTP, since it makes the protocol more efficient and flexible.

Figure 2.17 *FTP*

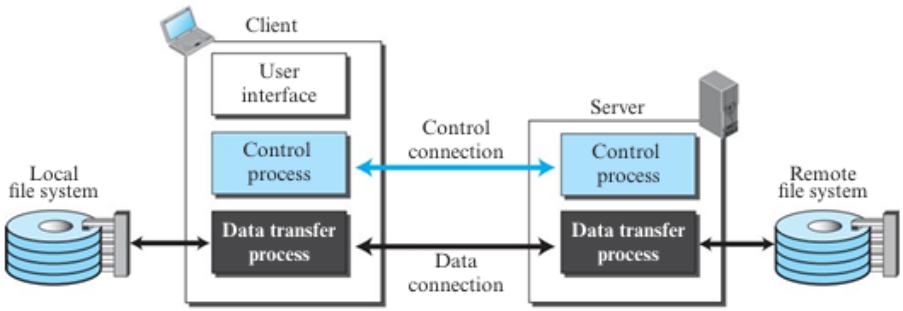


Figure: FTP architecture — separate control connection and data connection between client and server.

The Control Connection in FTP remains open throughout the entire FTP session and is used for sending commands and receiving responses; it uses the NVT ASCII character set, and each command or response ends with a CRLF sequence. FTP uses two well-known port numbers: port 21 for the control connection and port 20 for the data connection. The Data Connection, by contrast, is opened only when an actual file transfer is required and is closed once the transfer is complete; it can be opened and closed multiple times during the course of a single FTP session, once for each file transferred.

Common FTP Commands

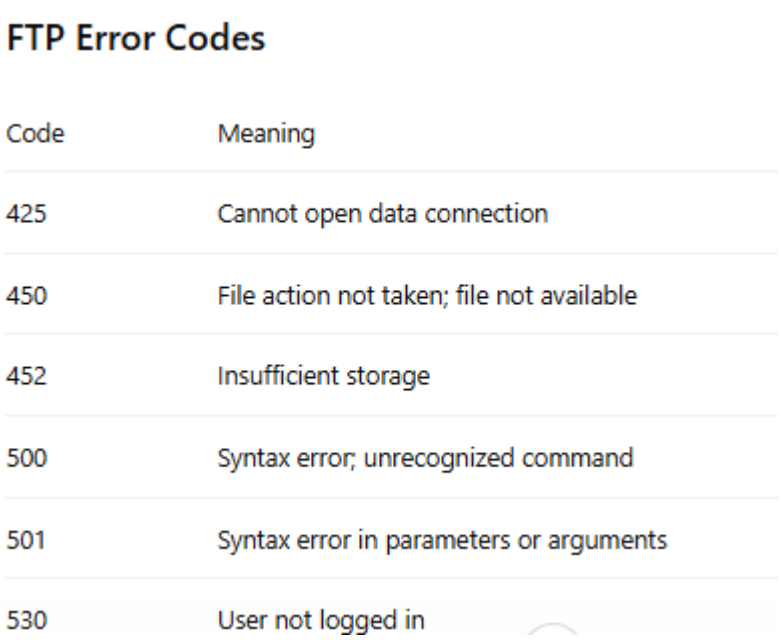
Command	Function
USER	Sends username
PASS	Sends password
LIST	Lists directory contents
RETR	Retrieves a file
STOR	Stores a file
DELE	Deletes a file
QUIT	Ends FTP session
PORT	Specifies client port for data transfer

Figure: Lifetimes of the FTP control connection versus the data connection during a session.

The establishment of the data connection proceeds as follows: the client performs a passive open using an ephemeral port; the client then sends this port number to the server using the PORT command; and the server performs an active open using its own port 20 together with the client's ephemeral port. Before any file transfer takes place, FTP also defines three additional characteristics: the Data Structure, the File Type, and the Transmission Mode.

Regarding data structure, FTP supports a File Structure (the default), which treats the file as a continuous stream of bytes; a Record Structure, in which the file is divided into records; and a Page Structure, in which the file is divided into numbered pages. Regarding file types, FTP supports ASCII files, EBCDIC files, and Image (binary) files. Regarding transmission modes, FTP supports Stream Mode (the default), which transfers a continuous stream of bytes; Block Mode, in which data is transferred in blocks with headers; and Compressed Mode, in which data is transferred in a compressed form to save bandwidth.

FTP supports three basic file transfer operations: Retrieve, in which a file is transferred from the server to the client; Store, in which a file is transferred from the client to the server; and Directory Listing, in which directory information is sent from the server to the client so the user can see the available files.



FTP Error Codes

Code	Meaning
425	Cannot open data connection
450	File action not taken; file not available
452	Insufficient storage
500	Syntax error; unrecognized command
501	Syntax error in parameters or arguments
530	User not logged in

Figure: The three basic FTP operations — Retrieve, Store, and Directory Listing.

Electronic Mail (E-Mail)

Electronic Mail (e-mail) is an application-layer service that allows users to exchange messages over a computer network or the Internet. Unlike applications such as HTTP and FTP, in which a server continuously waits for requests from clients and provides an immediate response, e-mail works quite differently. E-mail is generally considered a one-way transaction, meaning that when one user sends a

message to another, an immediate response is not mandatory; the receiver may choose to reply, or may not respond at all, and if a reply is eventually sent, it is treated as a separate, independent one-way transaction of its own.

In a typical e-mail system, it would be neither practical nor logical for every user to run a server program continuously and wait for incoming messages at all times, since users routinely switch off their computers or disconnect from the network when they are not actively using them. To overcome this limitation, e-mail communication relies on intermediate mail servers, which store, forward, and deliver messages between users. The sender's e-mail client sends the message to a mail server, which then transfers it onward to the recipient's mail server; when the recipient eventually becomes available, they can access and retrieve the message using their own e-mail client.

Thus, e-mail implements the client-server model in an indirect manner: users run only client programs whenever they need to send or receive messages, while dedicated mail servers handle the actual storage and delivery of e-mails on their behalf. This approach allows reliable communication to take place even when the sender and receiver are not online at the same time.

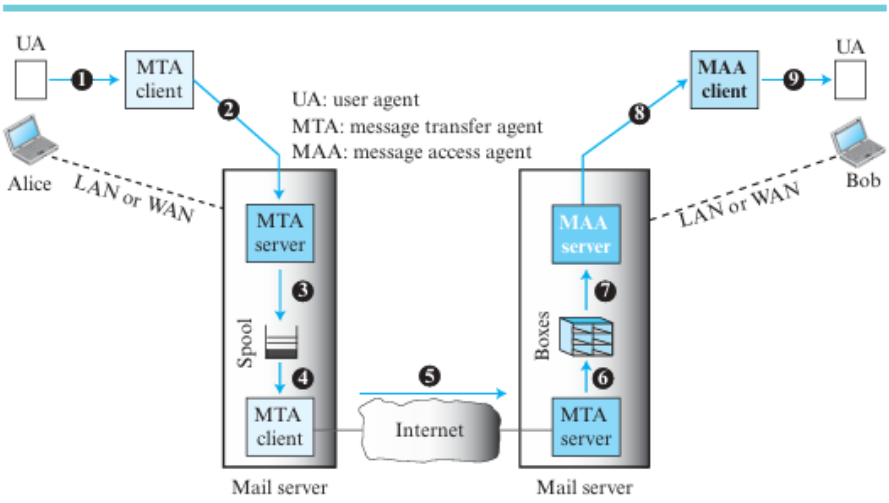


Figure: Indirect client-server model used by e-mail — intermediate mail servers store and forward messages.

Architecture of E-Mail

The architecture of an e-mail system consists of users, mail servers, mailboxes, queues, and different software agents that all work together to send and receive messages. In a typical e-mail scenario, the sender (commonly named Alice in textbook examples) and the receiver (commonly named Bob) are each connected to their respective mail servers through a LAN or WAN. Each user has a mailbox on their mail server, where received messages are stored, and a queue, or spool, where outgoing messages wait before they are actually sent.

When Alice wants to send an e-mail, she uses a User Agent (UA) to compose and submit the message to her own mail server. The message is then placed in a queue and subsequently transferred across the Internet using a Mail Transfer Agent (MTA). The MTA follows the client-server model, in which the client transfers the message to the destination mail server, while the server runs continuously to accept incoming messages from many senders. Once the message reaches Bob's mail server, it is stored in Bob's mailbox. To actually read the message, Bob uses a Message Access Agent (MAA), which retrieves the message from the mail server. Unlike the MTA, which is described as a push protocol because it pushes messages toward the destination server, the MAA is described as a pull protocol because it allows the user to retrieve messages only when they choose to do so. This overall architecture eliminates the need for users to keep their computers running continuously and enables reliable e-mail communication even when the sender and receiver are never online at exactly the same time.

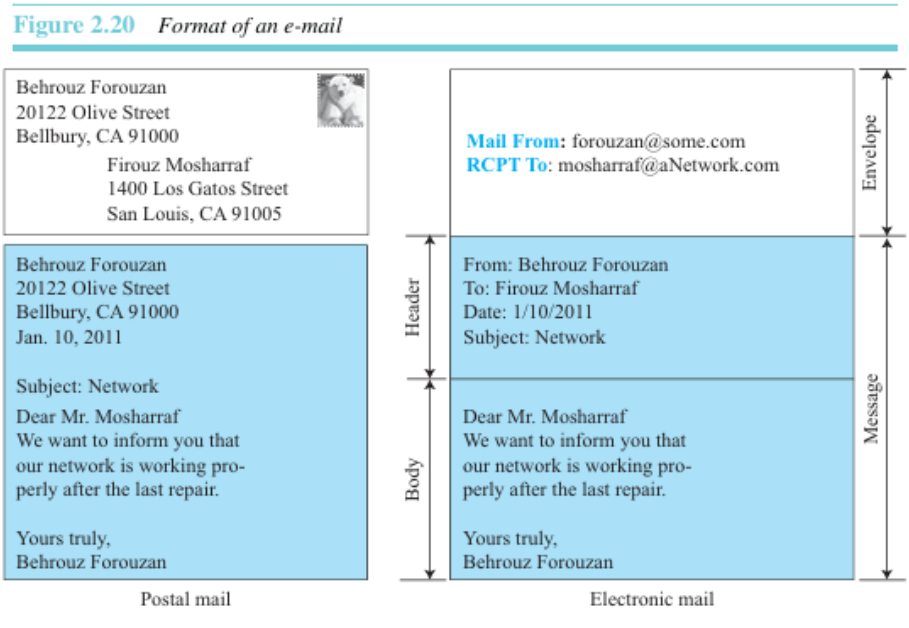


Figure: Architecture of an e-mail system — User Agents, mail servers, mailboxes, queues, MTA, and MAA.

User Agent (UA)

A User Agent (UA) is the first component of an electronic mail system and provides services to users for creating, sending, receiving, reading, replying to, and forwarding e-mail messages. It is a software program that acts as an interface between the user and the underlying e-mail system, and it also manages the local mailboxes stored on the user's own computer.

User agents can be broadly classified into two types: command-driven and GUI-based. Command-driven user agents were commonly used in the early days of e-mail and required users to enter simple keyboard commands to perform tasks; examples include mail, pine, and elm. Modern e-mail systems,

in contrast, use GUI-based user agents, which provide graphical elements such as icons, menus, buttons, and windows, making e-mail operations far easier and more user-friendly; examples include Microsoft Outlook and Eudora.

To send an e-mail, the user first creates the message using the User Agent. An e-mail is conceptually similar to postal mail and consists of an envelope and a message. The envelope contains information such as the sender's address, the receiver's address, and other delivery-related details. The message itself contains two parts: a header, which includes information such as the sender, receiver, subject, and other control details, and a body, which contains the actual content of the message. Once the message has been prepared, the User Agent sends it onward to the mail server for delivery.

When receiving e-mail, the User Agent is activated either directly by the user or automatically through a timer. If new mail is available in the mailbox, the User Agent notifies the user and displays a list of the received messages, showing summary information such as the sender's e-mail address, the subject line, and the date or time of receipt. The user can then select any message from this list and view its complete contents on the screen. In this way, the User Agent greatly simplifies the process of accessing and managing received e-mails.

Figure 2.21 *E-mail address*



Figure: Structure of an e-mail message — envelope, header, and body, as handled by the User Agent.

E-Mail Addresses and Mailing Lists

To ensure the proper delivery of e-mail messages, an e-mail system relies on a unique addressing scheme. On the Internet, an e-mail address consists of two parts separated by the "@" (at) symbol: the local part and the domain name. The local part identifies the user's mailbox, which is a special file where all incoming messages for that user are stored until they are retrieved. The domain name identifies the mail server, or mail exchanger, responsible for handling that user's e-mail; this domain name may be obtained from the Domain Name System (DNS) or may simply represent the name of an organization. This addressing scheme ensures that every e-mail message reaches its correct destination efficiently and accurately.

Electronic mail also supports mailing lists, also known as group lists, which allow a single name or alias to represent multiple e-mail addresses at once. Instead of sending the same message individually to several recipients, a user can simply send it to the mailing list alias. The e-mail system then checks an alias database and automatically expands the alias into all of the individual e-mail addresses associated with it, creating separate copies of the message for each member of the group and

forwarding them to the Mail Transfer Agent (MTA) for delivery. Mailing lists simplify communication with groups and make it far easier to send the same information to many recipients simultaneously.

Message Transfer Agent (MTA) and SMTP

The Message Transfer Agent (MTA) is responsible for transferring e-mail messages from the sender to the receiver through the relevant mail servers. In a typical e-mail exchange, three separate client-server interactions are actually involved: the first between the sender and the sender's own mail server, the second between the sender's mail server and the receiver's mail server, and the third between the receiver and the receiver's mail server. The first two of these interactions use a Message Transfer Agent (MTA), while the third instead uses a Message Access Agent (MAA).

The standard Internet protocol used by the MTA is the Simple Mail Transfer Protocol (SMTP). SMTP is actually used twice during the overall process of e-mail transmission: first, to transfer the message from the sender's User Agent to the sender's own mail server, and second, to transfer the message between the sender's mail server and the receiver's mail server. A different protocol, such as POP or IMAP, is used later by the receiver specifically to access and download the message from their mail server.

SMTP itself operates using a command-response mechanism. The MTA client sends commands to the MTA server, and the server replies with appropriate responses; each command and response is terminated by a two-character end-of-line sequence consisting of a carriage return (CR) and a line feed (LF). Common SMTP commands include HELO (used to identify the client), MAIL FROM (used to specify the sender's address), RCPT TO (used to specify the recipient's address), DATA (used to indicate the start of the actual message content), and QUIT (used to terminate the session). Responses are sent from the server back to the client in the form of three-digit status codes, often accompanied by additional textual information describing the result of the command. In this way, SMTP provides a reliable and standardized method for transferring e-mail messages across the Internet.

Mail Transfer Phases in SMTP

The transfer of an e-mail message using SMTP occurs in three distinct phases: Connection Establishment, Message Transfer, and Connection Termination.

During the Connection Establishment phase, the SMTP client first creates a TCP connection with the SMTP server using port 25. The server then sends back a 220 (Service Ready) response to indicate that it is ready to receive mail; if the service happens to be unavailable, the server instead sends a 421 response. Next, the client sends a HELO command, along with its own domain name, to identify itself to the server, and the server acknowledges this by sending a 250 (Request Completed) response, or another appropriate response code if there is a problem.

Once the connection has been established, the Message Transfer phase begins. The client sends a MAIL FROM command containing the sender's e-mail address, and the server responds with a 250 code if the command is accepted. The client then sends a RCPT TO command specifying the recipient's e-mail address, and the server again responds appropriately; if there are multiple recipients, this step is simply repeated once for each additional recipient. After all recipients have been specified, the client sends the DATA command to indicate the start of the actual message content, and the server responds with 354 (Start Mail Input), which allows the client to begin transmitting the message text.

The actual message is then sent line by line, with each line ending in a carriage return and line feed (CRLF) sequence; the end of the entire message is indicated by a line containing only a single period ("."). After receiving the complete message, the server sends back a 250 (OK) response to confirm successful receipt. Finally, in the Connection Termination phase, the client ends the SMTP session by sending the QUIT command, and the server responds with a 221 code to indicate that the connection is being closed, after which the underlying TCP connection is terminated. These three phases together ensure the reliable and orderly transfer of e-mail messages between mail servers on the Internet.

Domain Name System (DNS)

The Domain Name System (DNS) is an important client-server application within the TCP/IP protocol suite that helps other Internet applications by translating human-readable domain names into IP addresses. Although computers and networking devices actually communicate with one another using numerical IP addresses, people generally find it far easier to remember names such as "google.com" or "filesource.com." DNS therefore acts as a distributed directory service that maps domain names to their corresponding IP addresses, in much the same way that a telephone directory maps people's names to their telephone numbers.

Since the Internet contains billions of connected devices, maintaining a single, centralized directory for all of them would be both impractical and unreliable. Instead, DNS distributes its database across many DNS servers located all around the world. This distributed approach improves efficiency, scalability, and reliability, because users can access a nearby DNS server for faster resolution, and the failure of any single server does not bring down the entire naming system.

When a user wants to access a remote service, such as a file transfer server, the user typically knows only the server's domain name and not its actual IP address. The process begins when the user enters the host name into a client application, such as an FTP client. The client passes this host name to a DNS client on the local machine, which sends a query to a DNS server whose address is already known to the computer. The DNS server searches its database and returns the corresponding IP address of the requested host. The DNS client then provides this IP address to the application client, which finally uses it to establish an actual connection with the remote server. In this way, DNS serves as a vital

Internet service that allows users to access resources using easy-to-remember names, while still allowing the underlying TCP/IP protocols to operate purely in terms of numerical IP addresses.

Figure 2.35 *Purpose of DNS*

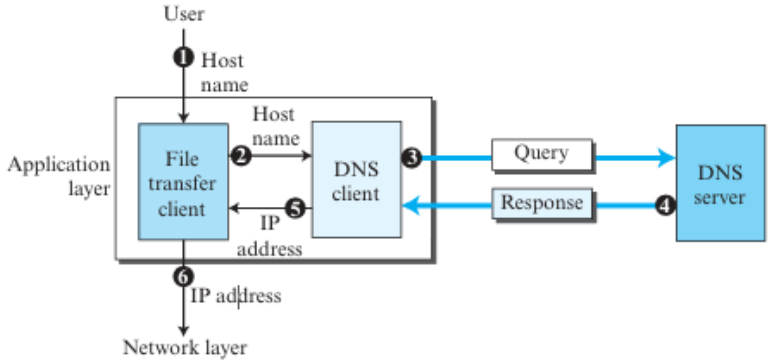


Figure: DNS name resolution — a client queries a DNS server to translate a domain name into an IP address before contacting the destination server.

Before a client can connect to a remote server such as a file transfer server, the server's domain name must first be translated into an IP address using DNS. This means that at least two connections are required overall: the first between the DNS client and the DNS server, used purely for name resolution, and the second between the actual application client and application server, used for the real data transfer. In some cases, DNS may require additional connections to other DNS servers before the correct IP address can finally be found.

Name Space and Domain Name Space

A name space is a system used to assign unique names to devices connected to the Internet, so that each name corresponds to exactly one unique IP address. Name spaces can generally be organized as either flat or hierarchical. In a flat name space, each name is simply a sequence of characters with no internal structure, which requires strict central control to avoid duplicate names; this approach is not well suited to very large networks like the Internet. A hierarchical name space, in contrast, consists of multiple levels, which allows the responsibility for assigning names to be distributed among different organizations, making the system far more scalable while still ensuring that every name remains unique. For example, two different organizations can each use the same host name, such as "caesar," while their complete domain names — "caesar.first.com" and "caesar.second.com" — remain globally unique.

To implement this kind of hierarchical naming system, the Internet uses a Domain Name Space, which is organized as an inverted tree structure with a single root at the top. Each node in this tree has a label, which can contain up to 63 characters, and the labels of sibling nodes under the same parent must all be different from one another to guarantee uniqueness. Every node in the tree also has a domain name,

formed by concatenating the labels along the path from that node up toward the root, separated by dots. Domain names are conventionally read from the node itself toward the root. A complete domain name that ends with the root label (represented by a trailing dot) is called a fully qualified domain name (FQDN). This hierarchical structure allows DNS to efficiently organize and manage millions of domain names across the entire Internet.

Figure 2.36 *Domain name space*

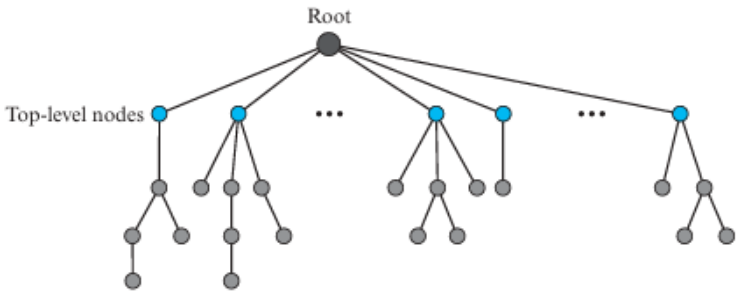


Figure: The Domain Name Space represented as an inverted tree, with the root at the top and labeled nodes below.

Figure 2.37 *Domain names and labels*

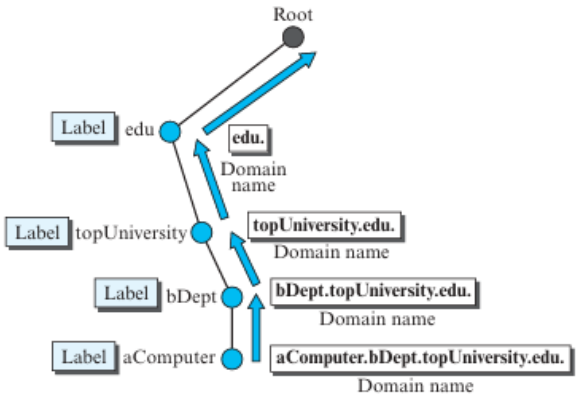


Figure: Formation of a domain name by concatenating labels from a node up to the root.

Figure 2.38 *Domains*

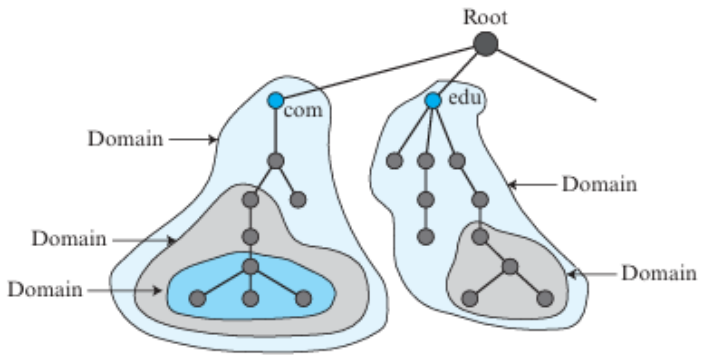


Figure: Example of a fully qualified domain name (FQDN) ending with the root label.

Distribution of the Name Space, Zones, and Servers

The DNS name space is distributed among many different DNS servers, because storing all domain information on a single server would be both inefficient and unreliable. To manage this large volume of data, the overall domain name space is divided into domains and subdomains, and different DNS servers are made responsible for different portions of this hierarchy. This creates a hierarchy of name servers, in which each server is authoritative for a particular domain or subdomain.

A zone is the specific portion of the DNS name space for which a particular DNS server is responsible. If a domain has not been divided into any subdomains, the domain and the zone are effectively the same thing. However, when a domain is divided and authority over parts of it is delegated to lower-level servers, the original server retains only partial information, while the detailed information about each subdomain is maintained by the servers specifically responsible for those subdomains. This distributed approach improves the overall efficiency, scalability, and reliability of the DNS system.

Figure 2.39 *Hierarchy of name servers*

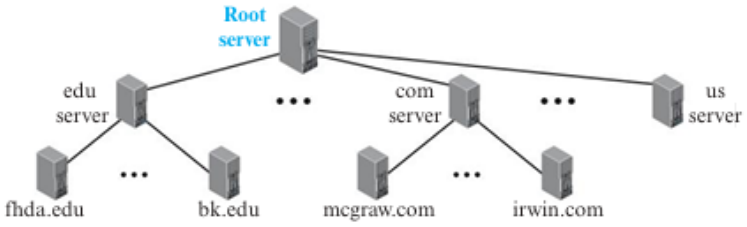


Figure: Division of the DNS name space into domains, subdomains, and zones of authority.

A Root Server is a DNS server whose zone of authority covers the entire domain name space. Rather than storing detailed information about every domain itself, a root server delegates authority to lower-level DNS servers and simply maintains references to them. Multiple root servers are deliberately

distributed across the world in order to improve the overall reliability and availability of the DNS system.

DNS uses two types of authoritative servers: Primary Servers and Secondary Servers. A Primary Server stores the original zone file and is responsible for creating, maintaining, and updating that file. A Secondary Server, by contrast, obtains a copy of the zone file from a primary server (or from another secondary server) and stores it locally; secondary servers cannot themselves modify the zone file, so any updates must be made on the primary server first and then transferred to the secondary servers. Both primary and secondary servers are considered authoritative for their respective zones. The purpose of secondary servers is to provide redundancy and improve overall reliability, ensuring that DNS services continue to function even if the primary server happens to fail. It is also possible for a single server to act as a primary server for one zone while simultaneously acting as a secondary server for a different zone.

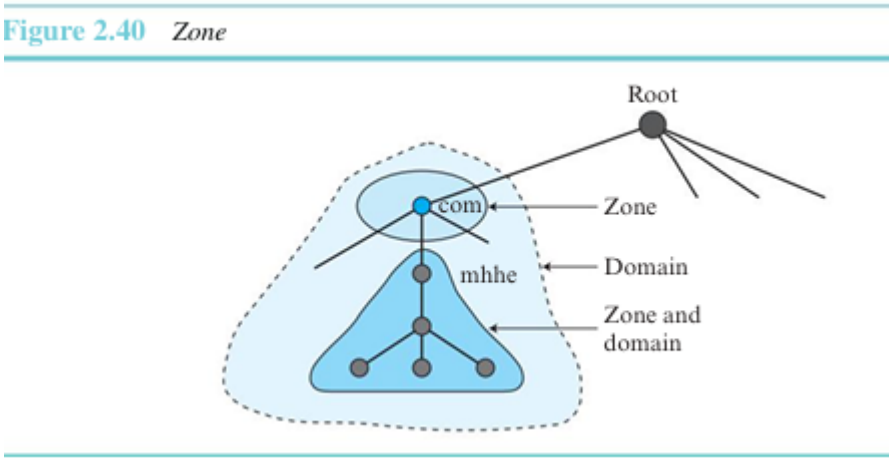


Figure: Relationship between a Primary DNS server and its Secondary (redundant) servers for a given zone.

DNS in the Internet: Generic and Country Domains

DNS is the protocol used to translate domain names into IP addresses on the Internet. Originally, the Internet domain name space was divided into Generic Domains, Country Domains, and Inverse Domains. However, inverse domains — which were used to find a hostname starting from a known IP address — have since been deprecated because of the rapid growth of the Internet. As a result, modern DNS mainly focuses on generic and country domains.

Generic Domains classify hosts based on their general purpose or the type of organization they belong to, and each domain in the DNS hierarchy effectively acts as an index into the overall domain name space database. Common generic domains include ".com" for commercial organizations, ".edu" for educational institutions, ".org" for non-profit organizations, ".gov" for government agencies, and ".net" for network service providers. These generic domains help organize and identify different types of

Figure 2.41 *Generic domains*

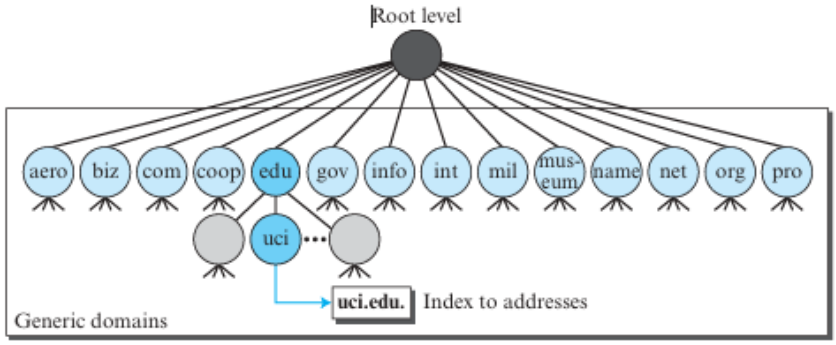


Figure: Classification of the DNS name space into generic domains, country domains, and (deprecated) inverse domains.

- Extra point: Country domains, such as ".in" for India, ".uk" for the United Kingdom, and ".us" for the United States, use a two-letter country code as defined by ISO 3166.
- Extra point: DNS resolution typically involves recursive and iterative queries between a local resolver, root servers, top-level domain (TLD) servers, and authoritative servers, culminating in the final IP address being returned to the client.

1.11 Peer-to-Peer (P2P) Networks

The Peer-to-Peer (P2P) paradigm is a network model in which computers communicate directly with one another without relying on a central server. Each computer participating in the network, called a peer, can act as both a client and a server at different times. The concept of P2P file sharing began with WWIVnet in 1987 and later evolved through a series of well-known systems, including Freenet (1999), Napster (1999), Gnutella (2000), Kazaa, BitTorrent, WinMX, and GNUnet (2001). P2P networks became especially popular for file sharing because they enable decentralized communication and resource sharing directly among users, without depending on any single central authority.

A Peer-to-Peer (P2P) network can be more precisely defined as a distributed network in which Internet users share resources directly with one another. When a peer possesses a file — such as an audio file, a video file, or a document — it makes that file available to other peers on the network. Any interested peer can then connect directly to the computer storing the file and download it; once downloaded, that peer can also share the same file with other peers, which increases the total number of copies available across the network over time. Because peers continuously join and leave the network, maintaining accurate information about which peers are currently active and which files they hold becomes an important ongoing challenge. Based on how this information is managed, P2P networks can be classified into centralized and decentralized networks.

Centralized P2P Networks

A Centralized Peer-to-Peer Network combines features of both the client-server and peer-to-peer paradigms, and is therefore often described as a hybrid P2P network. In this model, a central server maintains a directory containing information about the peers currently on the network and the files they are sharing, while the actual storage and transfer of files still takes place directly between the peers themselves. When a peer joins the network, it registers with the central server by providing its IP address along with a list of files it is willing to share. When another peer searches for a particular file, it sends a query to the central server; the server searches its directory and returns the IP addresses of the peers that possess the requested file. The requesting peer then connects directly to one of those peers and downloads the file, and the central directory is continuously updated as peers join and leave the network over time.

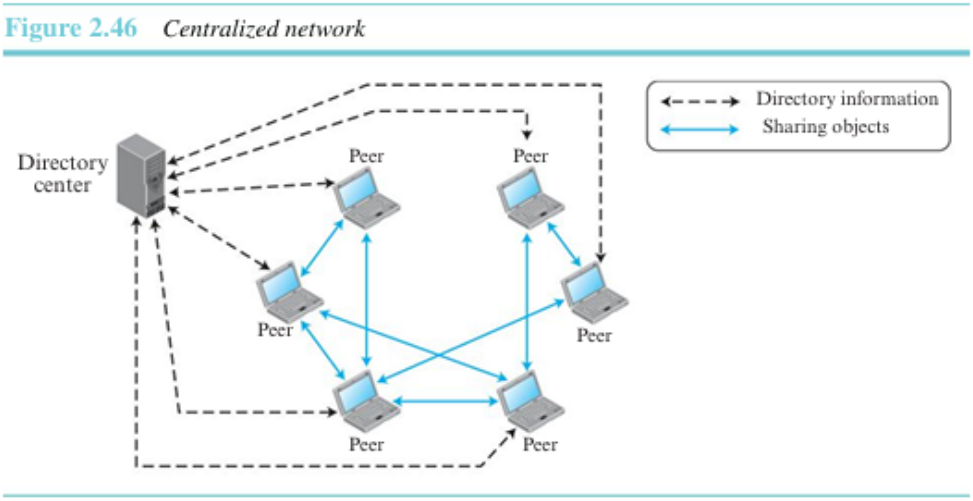


Figure: A centralized P2P network — a central server maintains the directory while file transfer occurs directly between peers.

Although centralized P2P networks simplify file searching and directory management considerably, they also carry several disadvantages. The central server can become a performance bottleneck due to heavy traffic, and it is also vulnerable to attacks and outright failures — if the central server becomes unavailable, the entire network may stop functioning altogether. Napster was the most well-known example of a centralized P2P network, and its reliance on central servers ultimately contributed to significant legal and operational challenges, eventually leading to its shutdown in 2001.

Decentralized P2P Networks

A Decentralized Peer-to-Peer Network does not rely on any central server or centralized directory system. Instead, the peers organize themselves into what is called an overlay network, which is a logical network built on top of the underlying physical network. Based on how peers are actually connected to one another within this overlay, decentralized networks can be further classified as either unstructured or structured.

In an Unstructured P2P Network, peers are connected to each other in a fairly random fashion. To locate a specific file, a search query is essentially flooded across the entire network, which can generate a large amount of unnecessary traffic and may still fail to find the desired file efficiently. Examples of unstructured P2P networks include Gnutella and Freenet.

In a Structured P2P Network, peers are instead connected according to a set of predefined rules, which enables considerably faster and more efficient file searches compared to the unstructured approach. These networks commonly rely on a Distributed Hash Table (DHT) to organize and locate resources across the network. DHT technology is widely used not only in P2P file-sharing applications but also in distributed databases, content distribution systems, and DNS. BitTorrent is a popular example of a structured P2P network that uses a DHT for efficient resource discovery and file sharing.

Case Study: BitTorrent

BitTorrent, designed by Bram Cohen, is one of the most widely used Peer-to-Peer file-sharing protocols in existence. Unlike traditional file-sharing systems, in which a single peer provides the entire file to a downloader, BitTorrent divides a large file into smaller pieces called chunks. Peers participating in the network simultaneously download and upload these chunks to one another, which makes overall file distribution significantly faster and more efficient than downloading from a single source.

The group of peers participating in the sharing of a particular file is called a swarm. A peer that possesses the complete file is known as a seed, while a peer that has downloaded only some parts of the file, and is still downloading the rest, is called a leech. As peers download individual chunks, they also simultaneously upload those same chunks to other peers, creating a genuinely collaborative sharing environment rather than a one-directional download.

In the original BitTorrent design, a central entity called a tracker manages the swarm for a particular file. When a new peer wants to download a file, it first obtains a torrent file, or metafile, containing information about the individual file pieces and the tracker's network address. The peer then contacts the tracker, which provides it with a list of neighboring peers already participating in that torrent. The new peer downloads the missing chunks from these peers and simultaneously uploads any chunks it already has available to others. Once all the chunks have been collected, the peer itself becomes a seed and can continue to help other users complete their downloads.

BitTorrent uses several strategies to ensure fairness and efficient utilization of network resources. Each peer maintains active connections with only a limited number of neighbors, typically around four, in order to avoid overloading the network. Peers are categorized as either unchoked (meaning they have active connections) or choked (meaning their connections are currently inactive); every few seconds, each peer evaluates its neighboring nodes and prefers to exchange data with those offering better

transfer rates. BitTorrent also uses a technique called optimistic unchoking, which periodically allows a brand-new peer to receive data even though it has not yet contributed anything to the swarm, helping new peers get started. Another important strategy is the rarest-first policy, in which peers prioritize downloading the least commonly available pieces first, which helps ensure a more balanced distribution of file chunks throughout the entire swarm.

One notable limitation of the original BitTorrent system was its dependence on a single tracker: if the tracker failed, new peers were unable to join the swarm at all. To overcome this problem, modern BitTorrent implementations instead use a Distributed Hash Table, particularly the Kademlia DHT algorithm, in which the tracking function is distributed among multiple peers rather than being concentrated on a single central server. A joining peer sends a request to the DHT network, which locates the responsible node and returns a list of peers already participating in that torrent. This eliminates the single point of failure that existed in the original design and significantly improves the scalability and reliability of the protocol.

In conclusion, BitTorrent is an efficient and highly scalable P2P file-sharing protocol that distributes the overall workload among all participating peers. By dividing files into small chunks and actively encouraging peers to both upload and download data simultaneously, BitTorrent achieves faster transfers and much better utilization of available resources compared to traditional download methods. The subsequent introduction of trackerless operation using DHT further enhanced its reliability by removing the network's dependence on any single central tracker.

- Extra point: BitTorrent clients typically use a "tit-for-tat" strategy at the chunk-exchange level, closely related to unchoking, to discourage free-riding (peers who only download without uploading).
- Extra point: Magnet links are a modern alternative to traditional .torrent files; they contain a cryptographic hash of the content and allow a peer to locate the swarm directly through the DHT without needing to download a separate metafile first.

Part A – Short Answer Questions (2 Marks)

- Define a computer network.
- List the advantages of computer networks.
- What are end devices? Give examples.
- What are connecting devices in a network?
- Define transmission media.
- Differentiate between wired and wireless media.
- What is a PAN?

- What is a LAN?
- What is a MAN?
- What is a WAN?
- Define an intranet.
- What is an internetwork?
- What is the role of a router?
- What is the role of a switch?
- Define Point-to-Point WAN.
- Define Switched WAN.
- Differentiate between Internet and Intranet.
- What is circuit switching?
- What is packet switching?
- Define ISP (Internet Service Provider).
- What is DSL?
- What is the TCP/IP protocol suite?
- Why does TCP/IP use layering?
- List the layers of the TCP/IP model.
- What is encapsulation?
- What is decapsulation?
- Define multiplexing.
- Define demultiplexing.
- What is the client-server paradigm?
- What is the peer-to-peer paradigm?

Short Essay Questions (5 Marks)

- Explain the need for computer networks.
- Describe the components of a computer network.
- Explain different types of transmission media.
- Compare LAN, MAN, and WAN.
- Explain the concept of an internetwork.
- Discuss the role of routers and switches in networking.
- Explain Point-to-Point WAN and Switched WAN.
- Differentiate between Internet and Intranet.
- Explain circuit switching with an example.

- Compare circuit-switched and packet-switched networks.
- Describe the structure of the Internet.
- Explain the different types of ISPs.
- Discuss various methods of accessing the Internet.
- Explain the TCP/IP protocol suite.
- Discuss the advantages of layering in TCP/IP.
- Explain the functions of the TCP/IP layers.
- Describe encapsulation and decapsulation.
- Explain multiplexing and demultiplexing.
- Compare Client–Server and Peer-to-Peer paradigms.

Long Answer Questions (10 Marks)

- Explain the components of a computer network with suitable diagrams.
- Discuss different types of computer networks (PAN, LAN, CAN, MAN, WAN, GAN).
- Explain the concept of internetworking and the role of routers and switches.
- Describe Point-to-Point WAN and Switched WAN with examples.
- Compare circuit switching and packet switching in detail.
- Explain the structure of the Internet and the role of backbone, provider, and customer networks.
- Discuss the functions and types of Internet Service Providers (ISPs).
- Explain various methods used to access the Internet.
- Describe the TCP/IP protocol suite and its layered architecture.
- Explain the responsibilities of each layer in the TCP/IP model.
- Explain logical communication in the TCP/IP protocol suite.
- Discuss encapsulation and decapsulation with neat diagrams.
- Explain multiplexing and demultiplexing with suitable examples.
- Describe the client-server paradigm and its applications.
- Explain the peer-to-peer paradigm, its advantages, disadvantages, and applications.

Important University-Level Questions

- Explain the layered architecture of the TCP/IP protocol suite.
- Compare circuit switching and packet switching.
- Explain the structure of the Internet.
- Discuss the role of routers and switches in internetworking.
- Explain encapsulation and decapsulation in TCP/IP.

- Describe multiplexing and demultiplexing.
- Compare client-server and peer-to-peer paradigms.
- Explain different methods of Internet access.
- Discuss the various types of computer networks.
- Explain the components of a computer network.

Case Study Questions

- Case Study 1: College Network — Design and explain a network suitable for a college campus with multiple departments.
- Case Study 2: Online Video Streaming — Explain the protocols and network requirements for a video streaming service.
- Case Study 3: Office Intranet — Explain how an intranet can be designed and secured for internal office communication.